

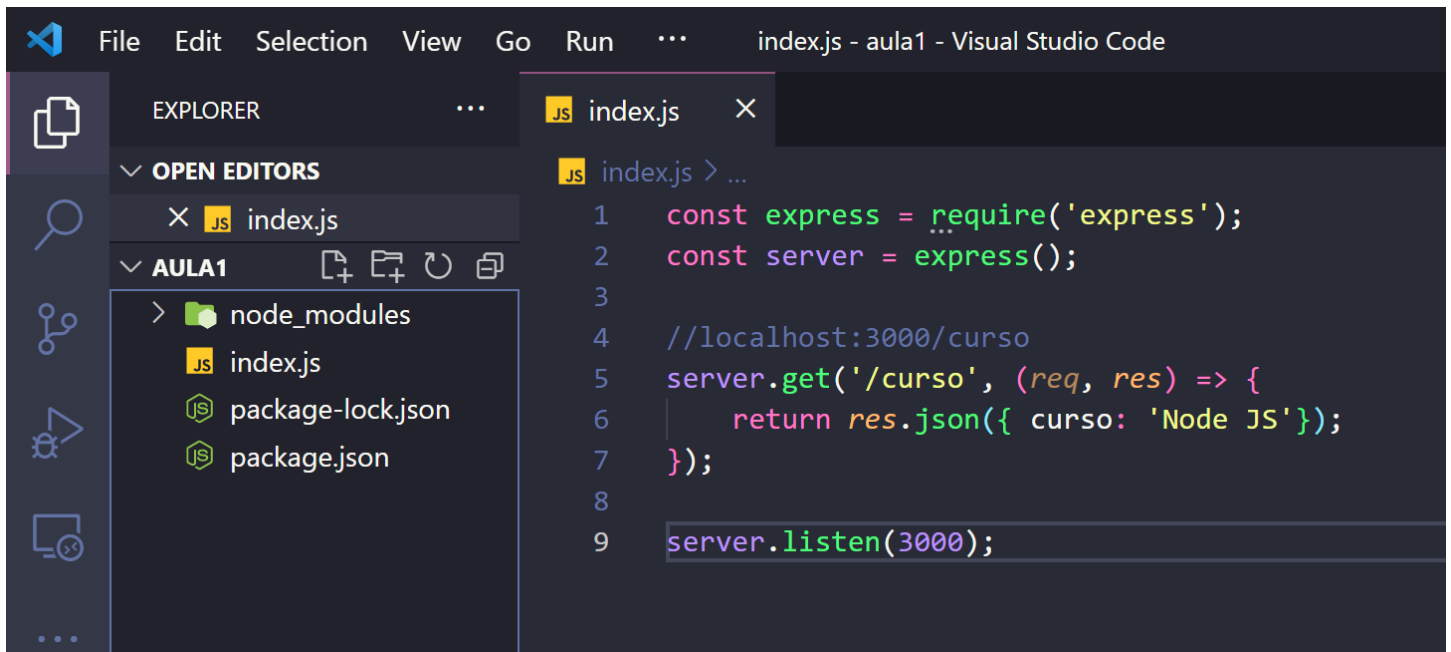
Aula 2 – CRUD com API

Objetivo da aula:

- Construindo APIs
- Entender query e route params
- Nodemon
- CRUD em Node JS
- Exercício: Fazendo o deploy e testando o CRUD via API Pública

Aplicação Node atual

1 – Na última aula montamos um projeto inicial em Node e criamos uma API simples, até aqui temos o seguinte código:

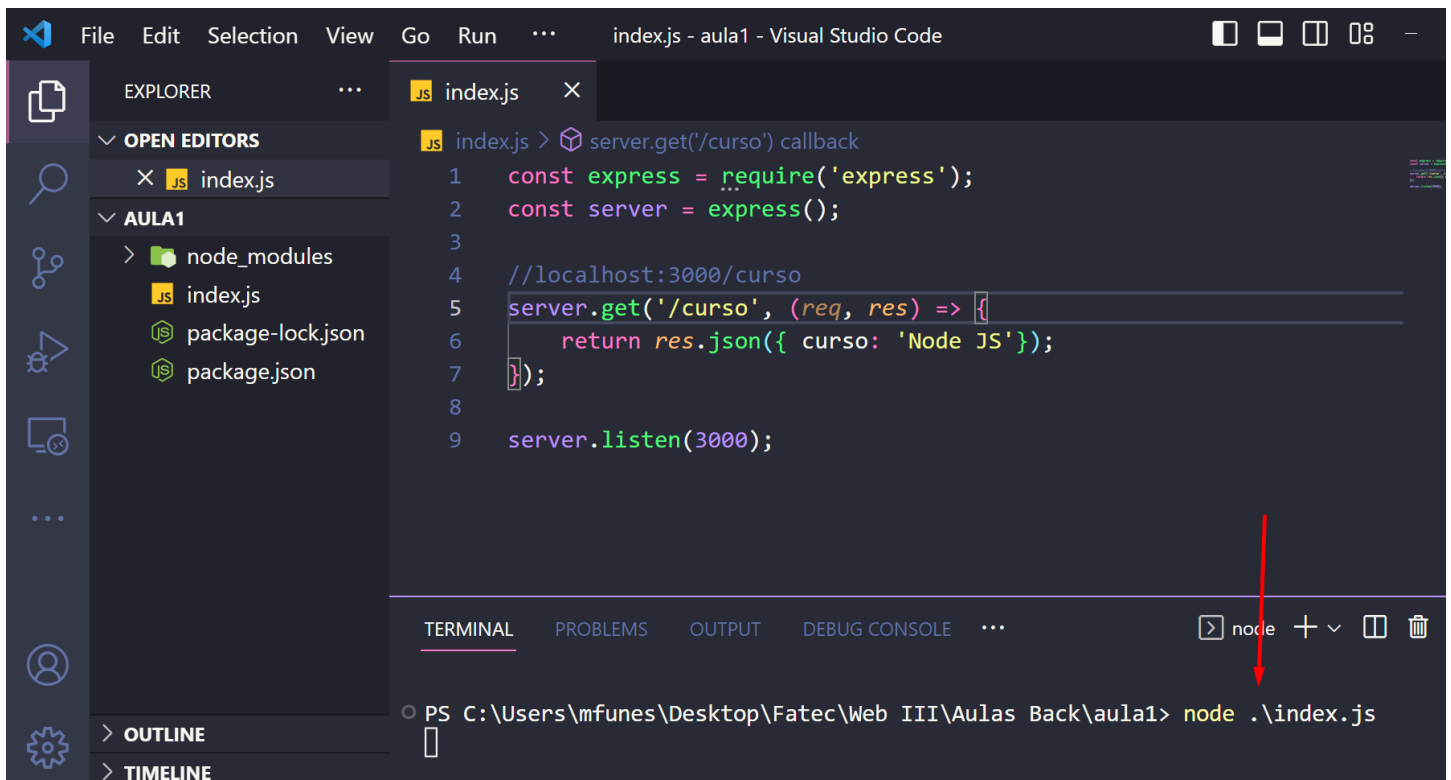


The screenshot shows the Visual Studio Code interface with the following components:

- Explorer Panel:** Displays the file structure of the project 'AULA1'. It includes a folder 'node_modules' and files 'index.js', 'package-lock.json', and 'package.json'.
- Open Editors Panel:** Shows the 'index.js' file is open in the editor.
- Code Editor:** Contains the following JavaScript code:

```
1  const express = require('express');
2  const server = express();
3
4  //localhost:3000/curso
5  server.get('/curso', (req, res) => {
6    return res.json({ curso: 'Node JS'});
7  });
8
9  server.listen(3000);
```

02 – Dê o comando “node index.js” para que nosso servidor possa ser executado e esperar algum cliente requisitar algo.



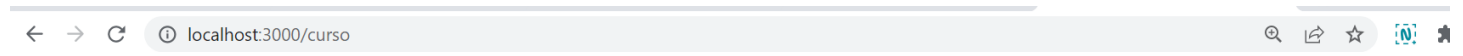
The image shows the Visual Studio Code interface with a file named `index.js` open in the editor. The file contains the following code:

```
server.get('/curso') callback
1  const express = require('express');
2  const server = express();
3
4  //localhost:3000/curso
5  server.get('/curso', (req, res) => {
6      return res.json({ curso: 'Node JS'});
7  });
8
9  server.listen(3000);
```

The Explorer sidebar on the left shows the project structure with a folder named `AULA1` containing `node_modules`, `index.js`, `package-lock.json`, and `package.json`.

The TERMINAL panel at the bottom shows the command prompt with the command `node .\index.js` entered. A red arrow points to the `node` command in the terminal.

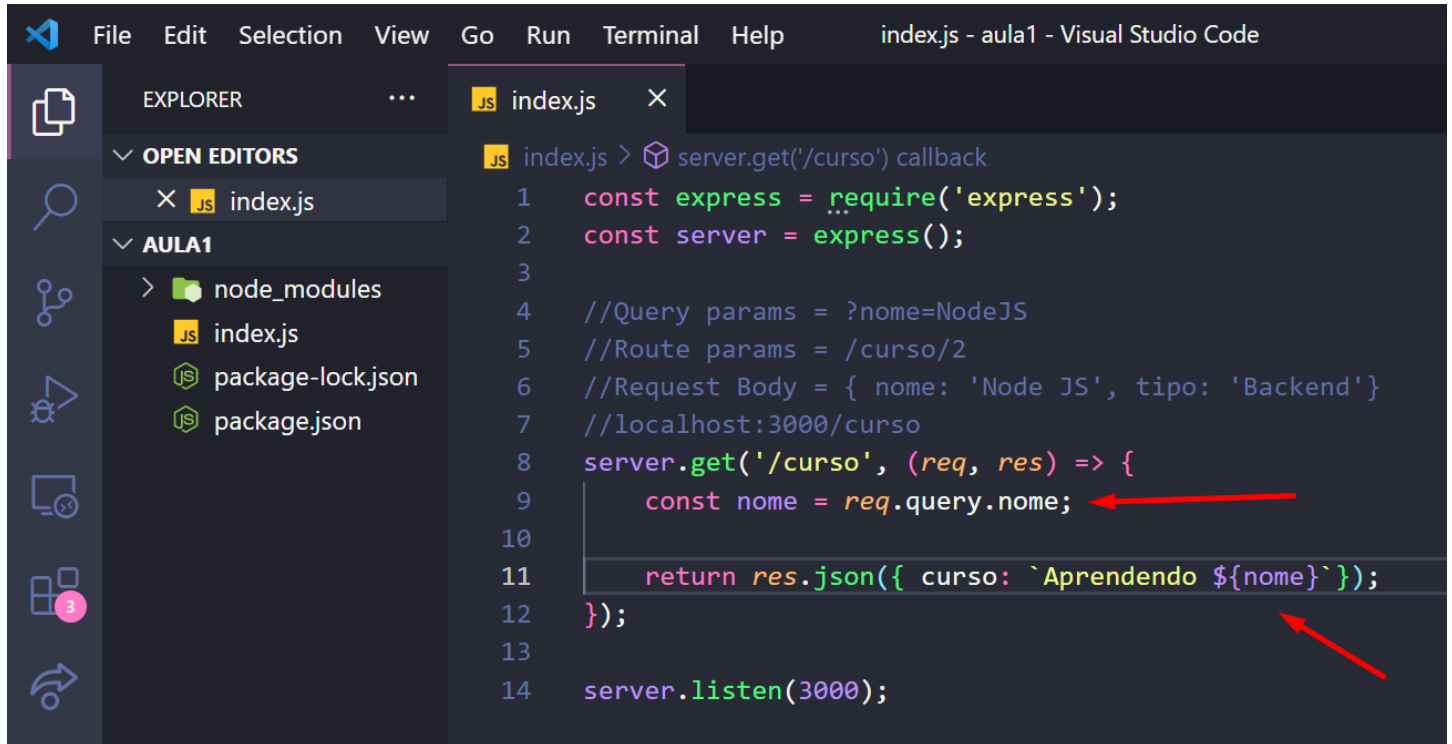
03 – No navegador acesse localhost:3000/curso e pronto, nosso back-end enviou ao front um JSON de exemplo



```
{"curso": "Node JS"}
```

Query Params

04 – Imagine que você deseja manipular as informações que existem nas rotas da requisição. Crie uma const chamada nome, vamos receber um parametro diretamente da Query da url digitada pelo usuário

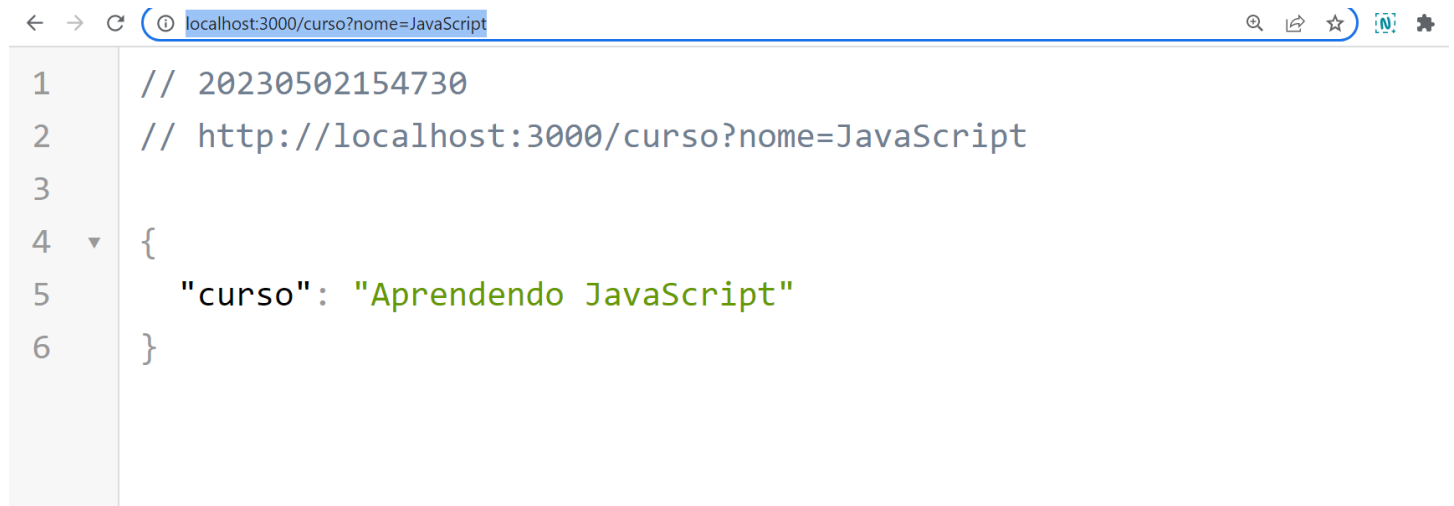


```
index.js - aula1 - Visual Studio Code

EXPLORER
  OPEN EDITORS
    index.js
  AULA1
    node_modules
    index.js
    package-lock.json
    package.json

index.js
1  const express = require('express');
2  const server = express();
3
4  //Query params = ?nome=NodeJS
5  //Route params = /curso/2
6  //Request Body = { nome: 'Node JS', tipo: 'Backend'}
7  //localhost:3000/curso
8  server.get('/curso', (req, res) => {
9    const nome = req.query.nome;
10
11    return res.json({ curso: `Aprendendo ${nome}` });
12  });
13
14  server.listen(3000);
```

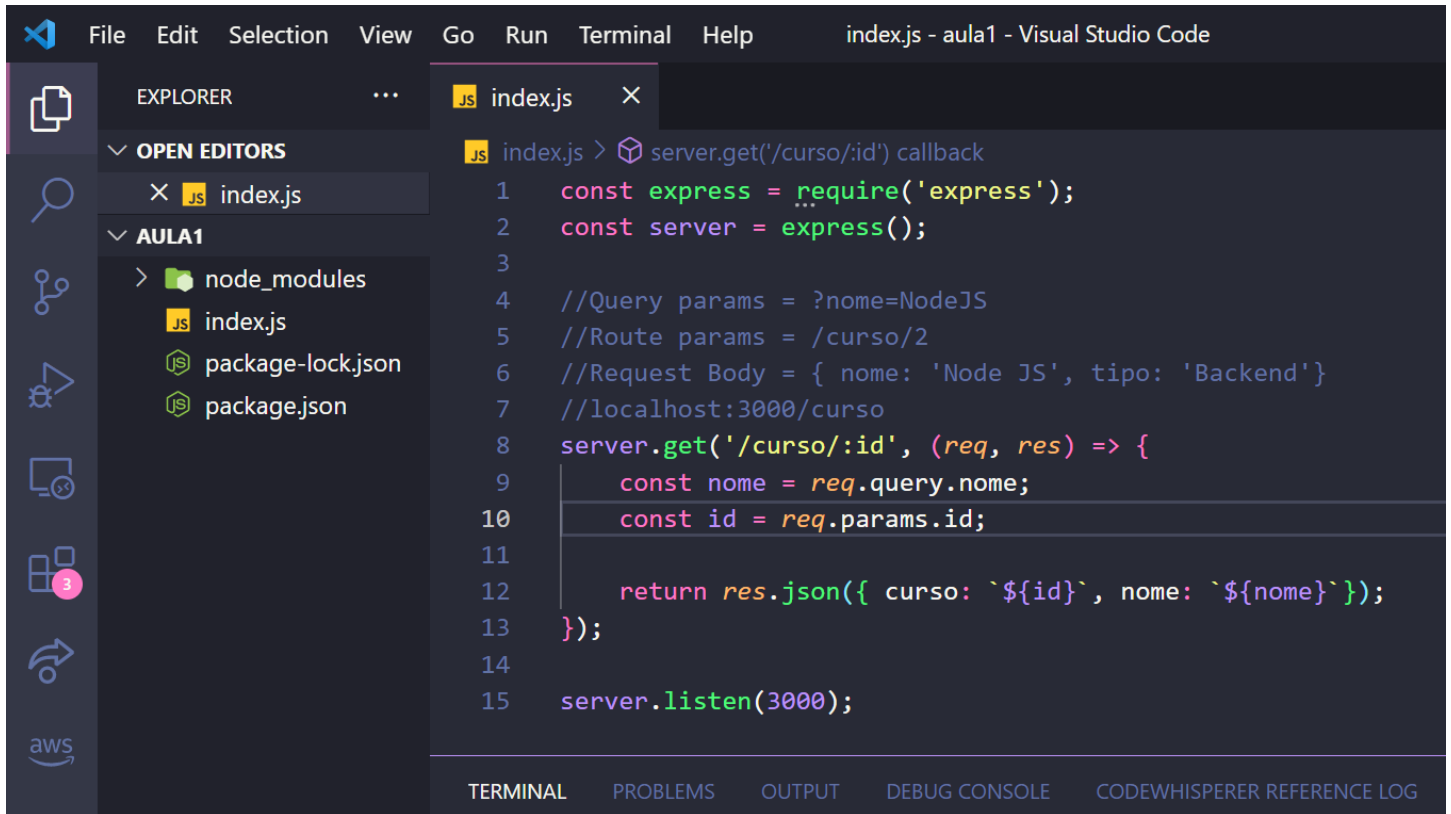
05 – No navegador passe agora o parametro nome e um valor para ele, exemplo: JavaScript. Veja que agora nosso back-end consegue receber esse valor e retornar para o cliente o mesmo valor.



The screenshot shows a web browser window with the address bar displaying `localhost:3000/curso?nome=JavaScript`. Below the address bar, a REST client interface shows a response for a GET request. The response status is `200` and the response body is a JSON object: `{ "curso": "Aprendendo JavaScript" }`. The response body is highlighted in green.

```
1 // 20230502154730
2 // http://localhost:3000/curso?nome=JavaScript
3
4 {
5   "curso": "Aprendendo JavaScript"
6 }
```

06 – Também podemos aplicar a mesma lógica nos Route params, ou seja, parametros que nos ajudam a manipular uma rota. Altere nossa rota para *curso/:id*, esse “:” indica que o back-end deve esperar que ali exista algum ID. Também crie uma const id para receber qual esse id.



```
File Edit Selection View Go Run Terminal Help index.js - aula1 - Visual Studio Code

EXPLORER
OPEN EDITORS
  X JS index.js
AULA1
  > node_modules
  JS index.js
  package-lock.json
  package.json

JS index.js > server.get('/curso/:id') callback
1  const express = require('express');
2  const server = express();
3
4  //Query params = ?nome=NodeJS
5  //Route params = /curso/2
6  //Request Body = { nome: 'Node JS', tipo: 'Backend'}
7  //localhost:3000/curso
8  server.get('/curso/:id', (req, res) => {
9    const nome = req.query.nome;
10   const id = req.params.id;
11
12   return res.json({ curso: `${id}`, nome: `${nome}` });
13 });
14
15 server.listen(3000);

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG
```

06 – Veja que agora conseguimos consultar tanto o “Route Params” (/curso/201) quanto uma “Query Params” que logo após o ? nos é passado nome=NodeJS. Apenas estamos pegando essa informação e enviando novamente ao cliente para testar.

```
localhost:3000/curso/201?nome=NodeJS

1 // 20230502160902
2 // http://localhost:3000/curso/201?nome=JavaScript
3
4 {
5   "curso": "201",
6   "nome": "JavaScript"
7 }
```


07 – Teste sua api no Reqbin e veja se temos o mesmo resultado que no navegador.

REQBIN

EXAMPLES SAVED

POST Request Example
Bearer Token Auth Example
HTML Form POST Example
GET Request Example
REST API POST Example
PUT Request Example
Auth Bearer Header Example
POST API Example
POST JSON Example
JSON Payload Example
Content-Length Example
REST API GET Example
Send Cookies Example
POST XML Example
JSON Response Example
POST JSON with Bearer Token

Curl POST Request Example
Curl GET Request Example
Curl POST JSON Example
Curl POST Body Example
Curl Basic Auth Example
Curl Bearer Token Example
Curl Download File Example
Curl Ignore SSL Checks

JavaScript POST Example
JavaScript GET Example
JavaScript Fetch JSON
JavaScript Array Join Example
JavaScript String Contains
JavaScript Sum Array

CurlPythonJavaScriptNode.jsPHPJavaJSONXMLContactAccount

Online REST & SOAP API Testing Tool

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

FileGenerate CodeTools

Status: 200 (OK)Time: 29 msSize: 0.03 kb

AuthorizationContentHeadersRaw (2)

Content (4)Headers (8)Raw (10)JSON

Token

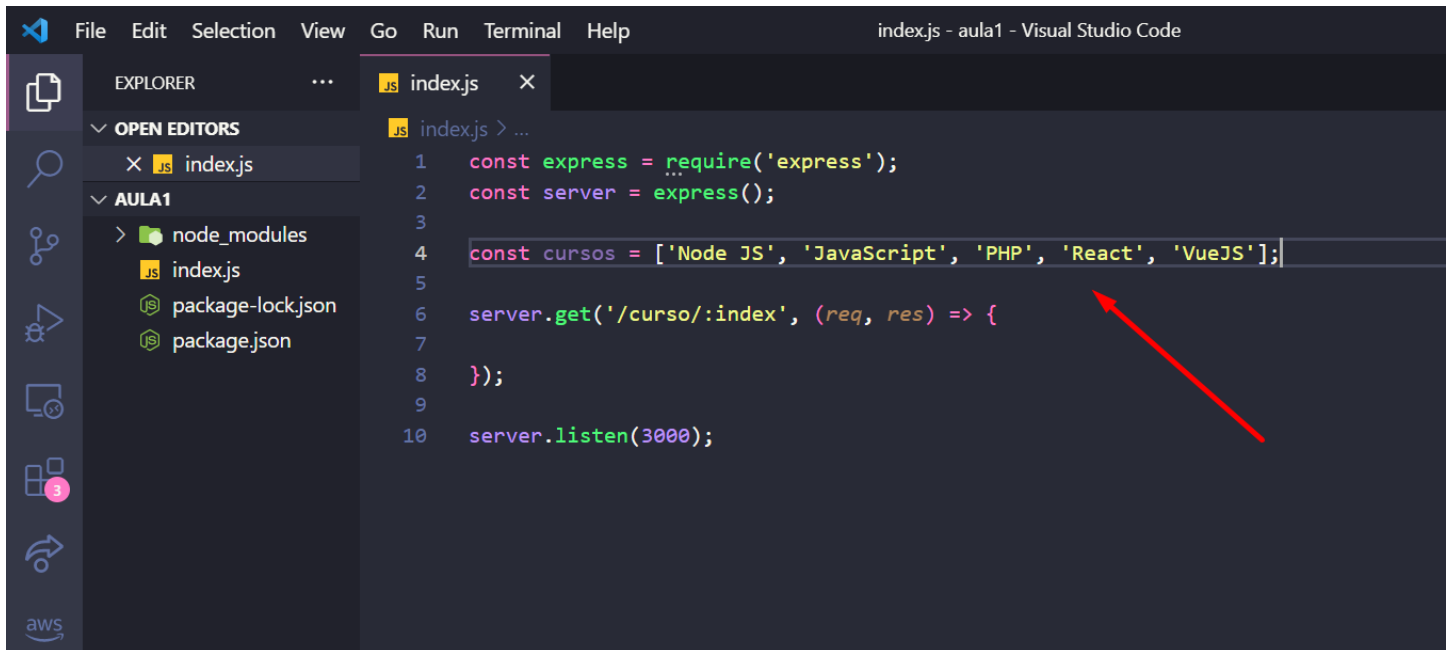
The authorization header will be automatically generated when you send the request. Read more about HTTP Authentication.

```
{  "cursor": "201",  "name": "NodeJS"}}
```

What is API?

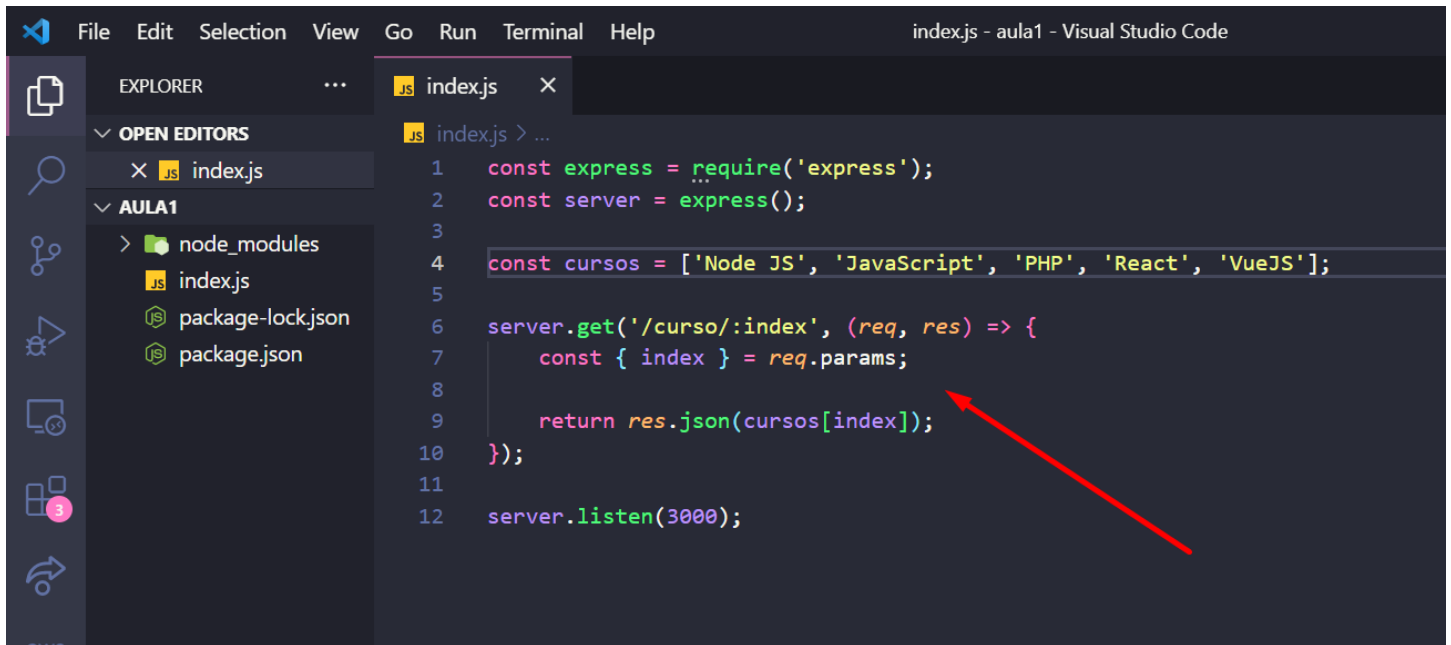
report this ad

08 – Imagine que no seu banco de dados exista um array de cursos que sua escola oferece, precisamos deixar o cliente requisitar informações de um curso via API. Para simular um banco, crie um array em JSON com vários cursos como na linha 4.



```
1 const express = require('express');
2 const server = express();
3
4 const cursos = ['Node JS', 'JavaScript', 'PHP', 'React', 'VueJS'];
5
6 server.get('/curso/:index', (req, res) => {
7
8 });
9
10 server.listen(3000);
```

09 – Agora como tratamos a requisição (req) do cliente e a nossa resposta (res)? Na rota passamos `/curso/:index` e como requisição esperamos que o cliente via API envie um valor, algo como: `/curso/3`. Caso ele envie este parametro, devemos responder “React” pois esse valor corresponde ao index 3 do nosso Array. (lembre que um Array começa no 0).



```
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React', 'VueJS'];
5
6  server.get('/curso/:index', (req, res) => {
7    const { index } = req.params;
8
9    return res.json(cursos[index]);
10 });
11
12 server.listen(3000);
```

10 – Teste agora sua API e passe o parametro 3 para verificar se de fato ele retorna o index correto.

The screenshot shows the ReqBin website, an online REST & SOAP API Testing Tool. The interface includes a sidebar with various example requests, a main area for creating and testing API requests, and a results panel on the right.

Request Details:

- URL: `http://localhost:3000/curso/3` (indicated by a red arrow)
- Method: `GET`
- Extension: `EXT`
- Status: `200 (OK)`
- Time: `6 ms`
- Size: `0.01 kb`

Response Details:

- Content (1): `"React"` (indicated by a red arrow)
- Headers (8)
- Raw (10)

Authentication:

- ☒ Bearer Token
- ☐ Basic Auth
- ☐ Custom
- ☐ No Auth

Token:

The authorization header will be automatically generated when you send the request. Read more about HTTP Authentication.

What is API?

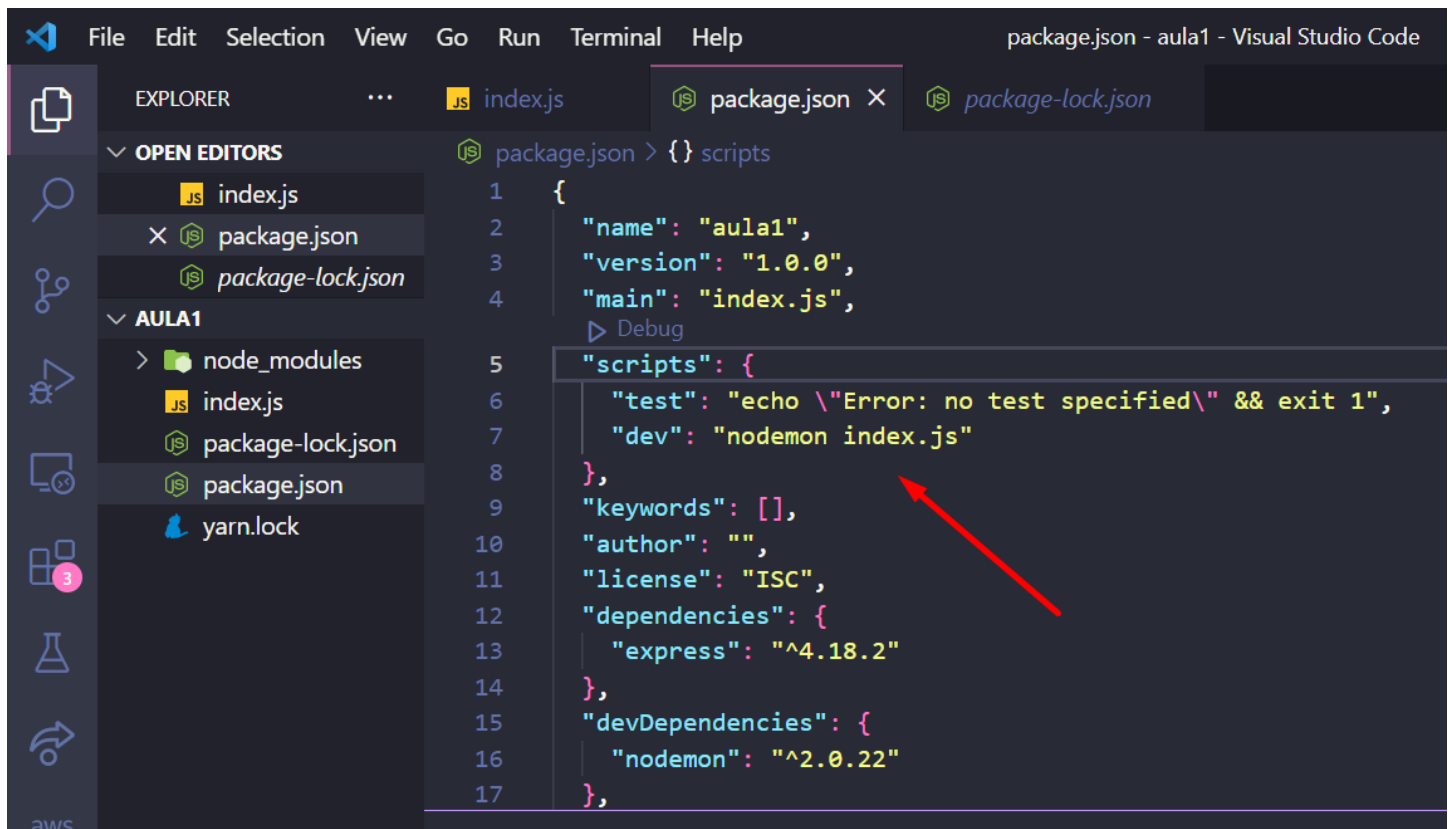
API (Application Programming Interface) is a computing interface that defines how software components interact with each other. It is a way of programmatically interacting with a separate

(Opcional)

11 – Para evitar de ficar toda vez tendo que parar e subir novamente nosso servidor node, existe o nodemon. Faça a instalação do nodemon com o seguinte comando:

```
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1> yarn add nodemon -D
```

12 – Vá em package.json e dentro de scripts adicione “dev”: “nodemon index.js”

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'AULA1' with a 'node_modules' directory and files like 'index.js', 'package-lock.json', 'package.json', and 'yarn.lock'. The main editor area displays the 'package.json' file. The 'scripts' section is expanded, showing the following JSON structure:

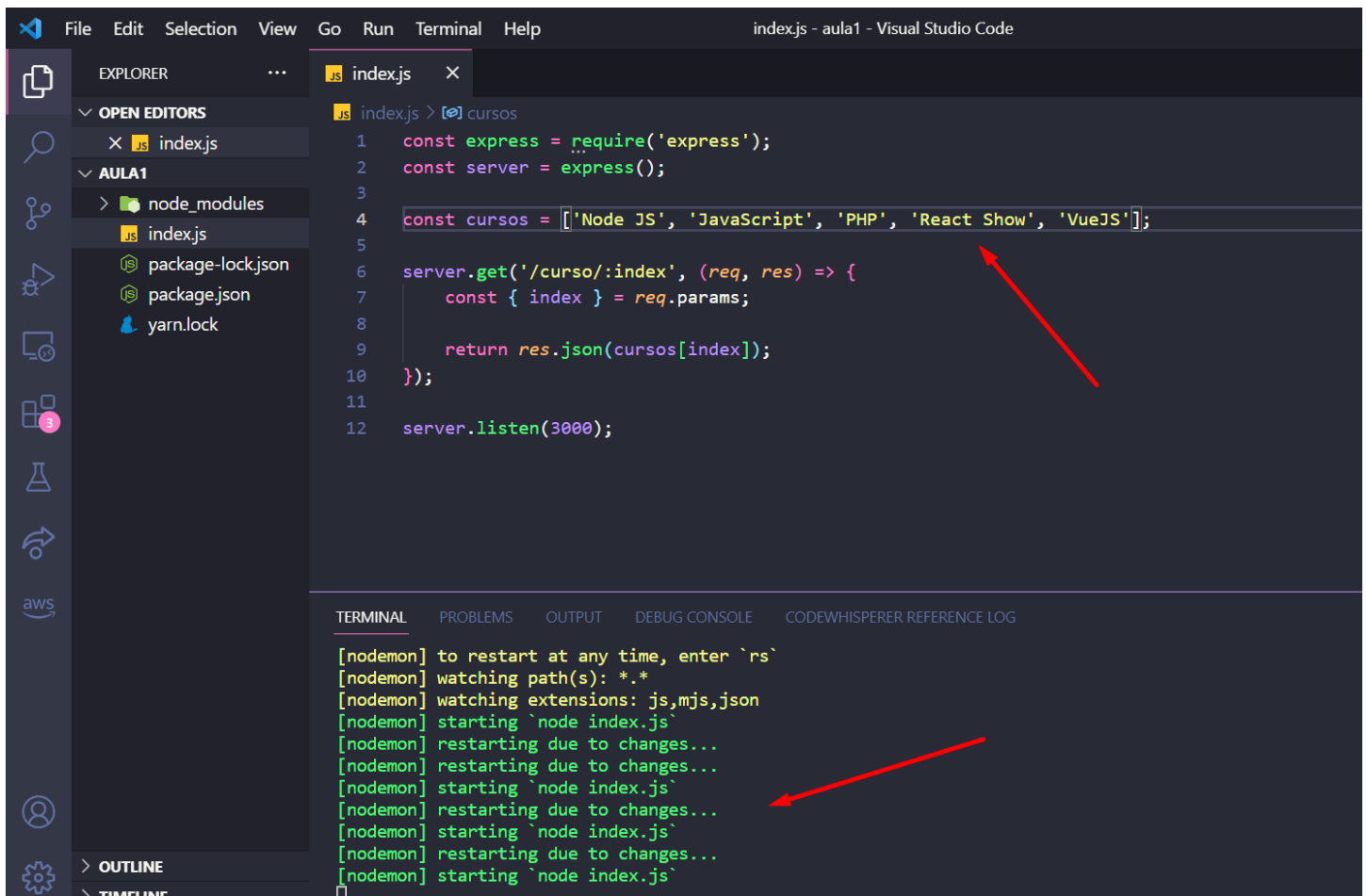
```
{
  "name": "aula1",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "nodemon index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

A red arrow points to the "dev" script entry. The top of the editor shows the menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help) and the title bar (package.json - aula1 - Visual Studio Code).

13 – Dê o comando yarn dev e pronto. Agora você pode alterar o index.js avontade que sempre o servidor irá ser atualizado :D

```
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1> yarn dev
```

14 – Veja que mudei para React Show e automaticamente ele atualizou, funcionando a API

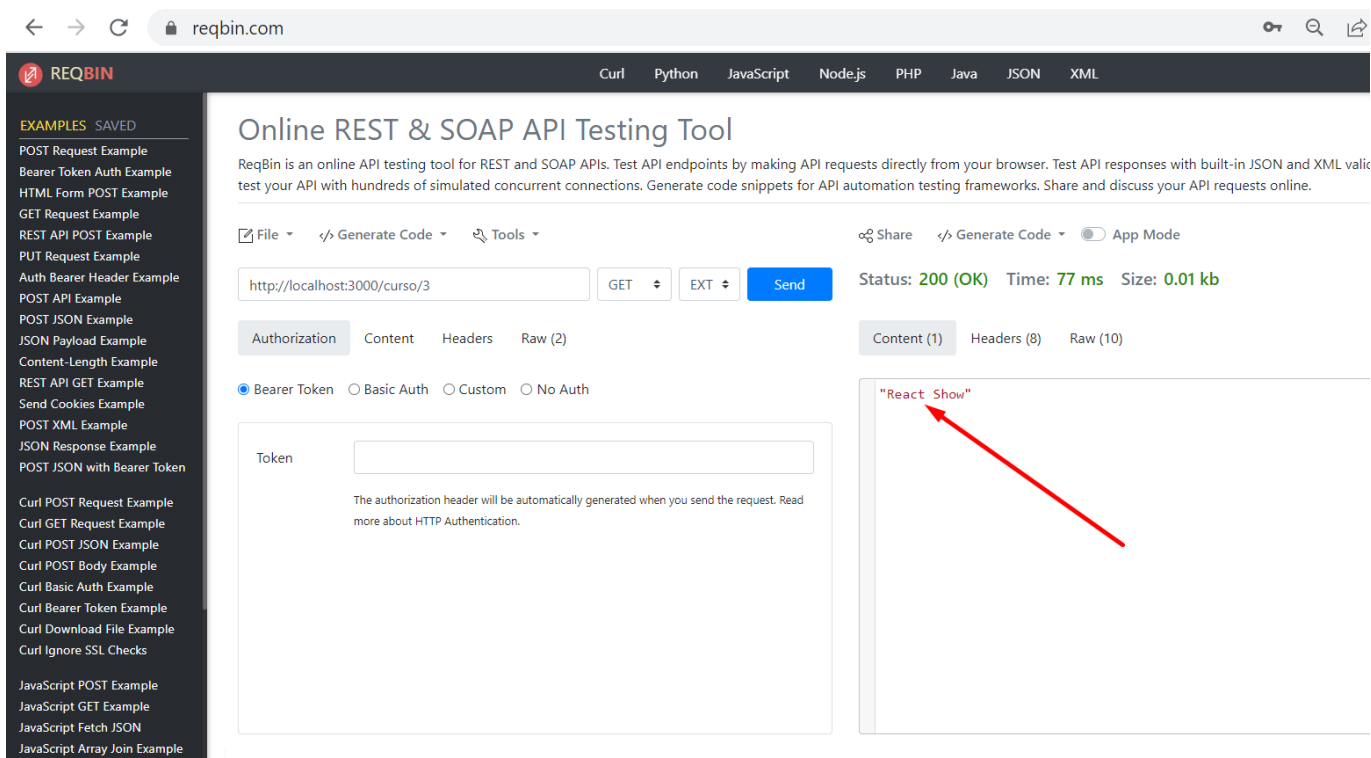


The screenshot shows the Visual Studio Code editor with the file `index.js` open. The code defines an Express server that serves a list of course names from the `curso` endpoint. The `curso` array includes 'Node JS', 'JavaScript', 'PHP', 'React Show', and 'VueJS'. A red arrow points to the 'React Show' entry in the array. The terminal at the bottom shows the output of the `nodemon` command, indicating that the server is restarting due to changes in the `index.js` file. Another red arrow points to the terminal output.

```
1 const express = require('express');
2 const server = express();
3
4 const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6 server.get('/curso/:index', (req, res) => {
7   const { index } = req.params;
8
9   return res.json(cursos[index]);
10 });
11
12 server.listen(3000);
```

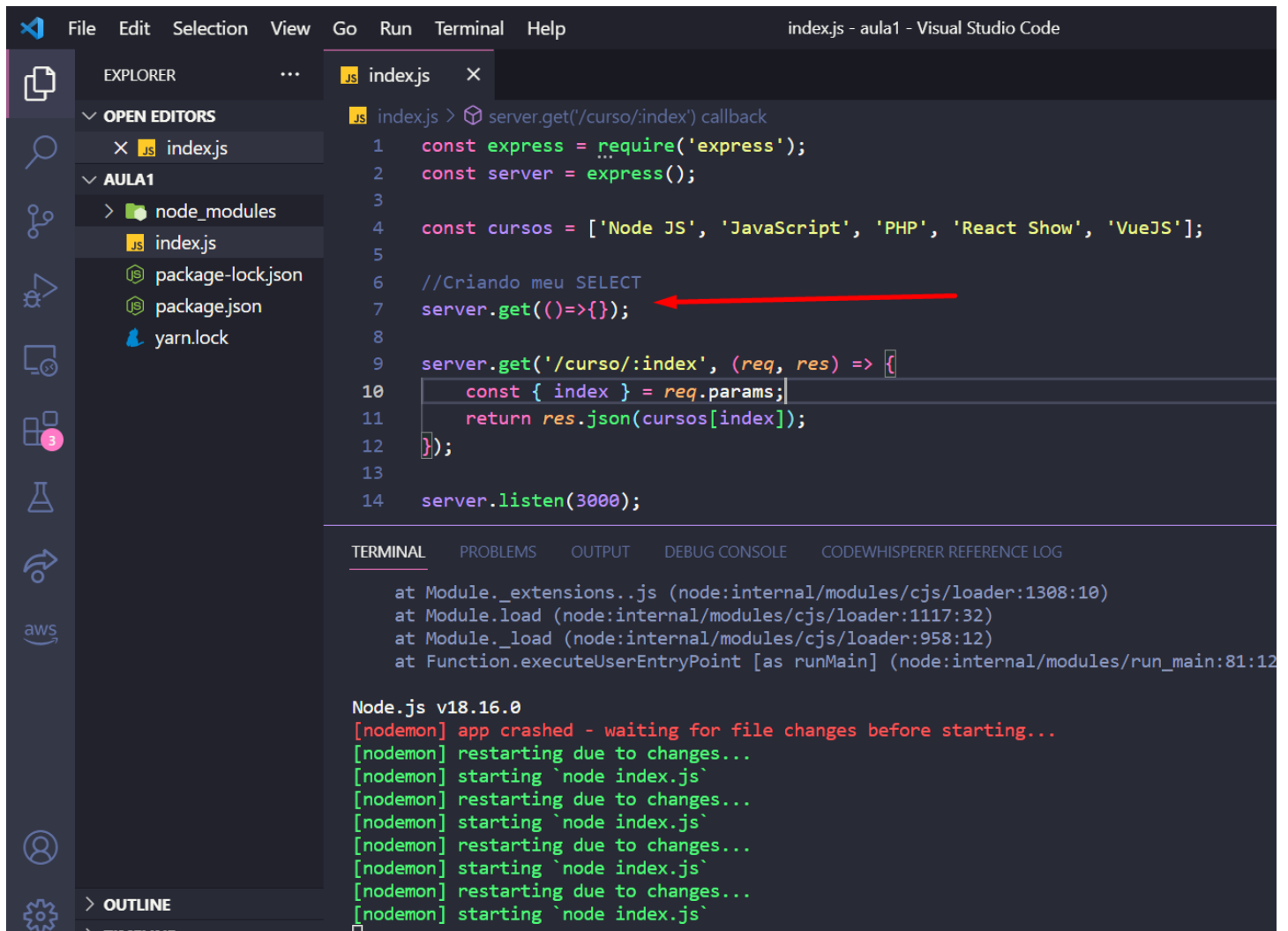
Terminal output:

```
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
```



CRUD em Node JS

15 – É comum nas API permitir a manipulação em CRUD, ou seja, que via API alguém possa adicionar, selecionar, atualizar ou excluir. Vamos começar permitindo que via API nosso Front-End consulte todos os nossos cursos. Crie um novo comando do tipo GET com `server.get(()=>{})`.



The screenshot shows the Visual Studio Code interface with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'aula1' with a file named 'index.js'. The code editor shows the following code:

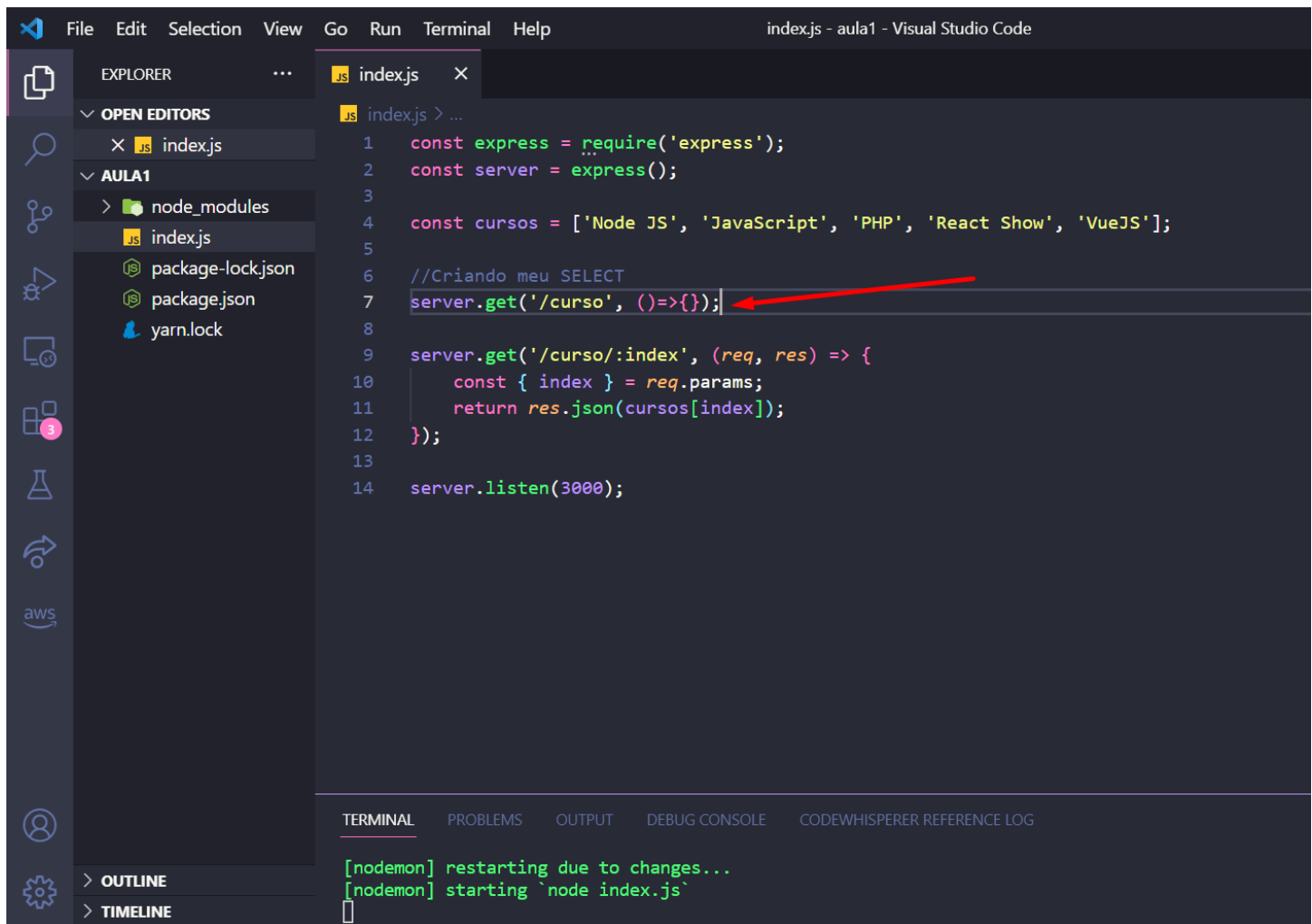
```
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6  //Criando meu SELECT
7  server.get(()=>{});
8
9  server.get('/curso/:index', (req, res) => {
10     const { index } = req.params;
11     return res.json(cursos[index]);
12 });
13
14 server.listen(3000);
```

A red arrow points to the `server.get(()=>{});` line in the code editor. Below the code editor, the terminal shows the following output:

```
at Module._extensions..js (node:internal/modules/cjs/loader:1308:10)
at Module.load (node:internal/modules/cjs/loader:1117:32)
at Module._load (node:internal/modules/cjs/loader:958:12)
at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)

Node.js v18.16.0
[nodemon] app crashed - waiting for file changes before starting...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
```

16 – Passe agora qual será a rota desse GET, nesse caso será /curso



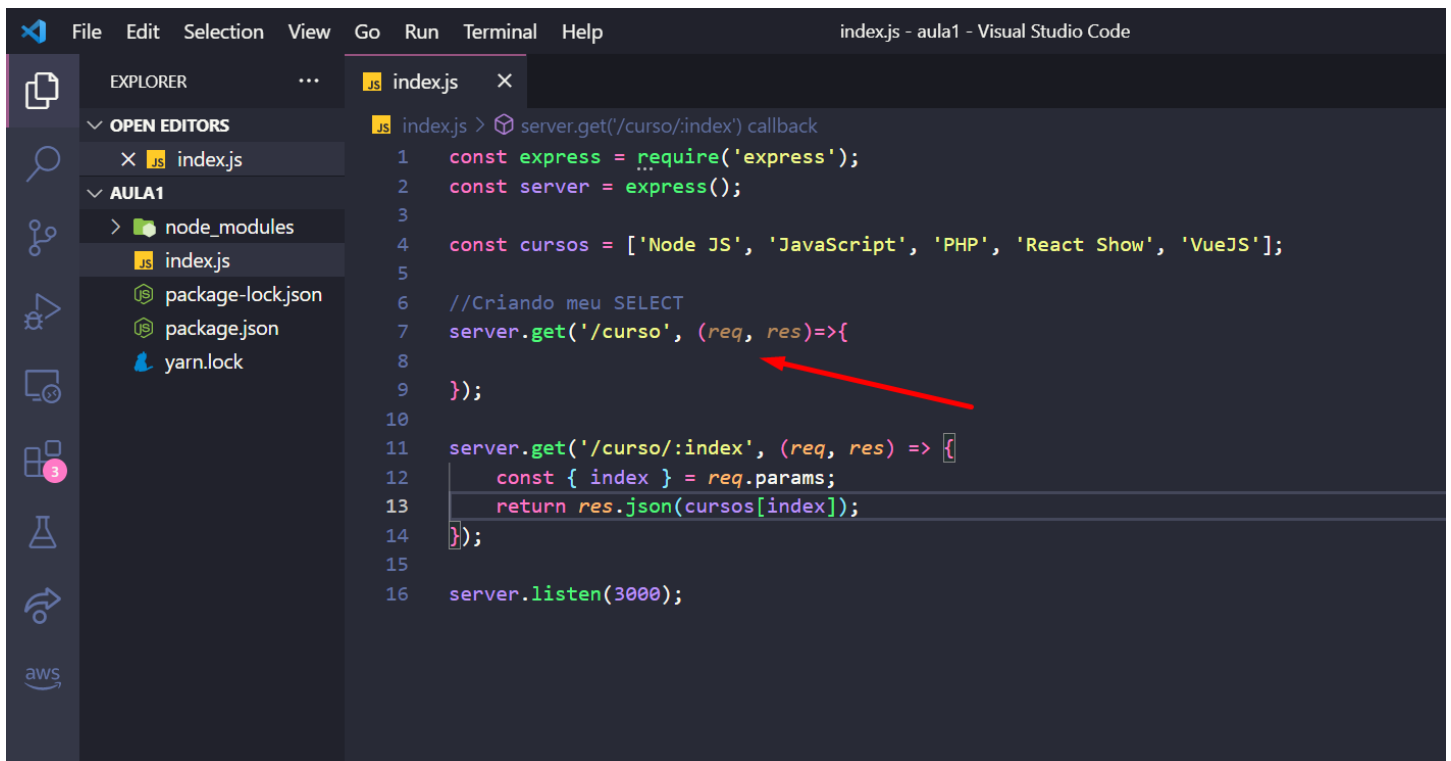
The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** Displays the file structure of the project, including `index.js`, `package-lock.json`, `package.json`, and `yarn.lock`.
- index.js:** The main file being edited, containing the following code:

```
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6  //Criando meu SELECT
7  server.get('/curso', ()=>{});
8
9  server.get('/curso/:index', (req, res) => {
10    const { index } = req.params;
11    return res.json(cursos[index]);
12  });
13
14  server.listen(3000);
```
- Terminal:** Shows the output of the command, indicating that the server is restarting and starting on port 3000:

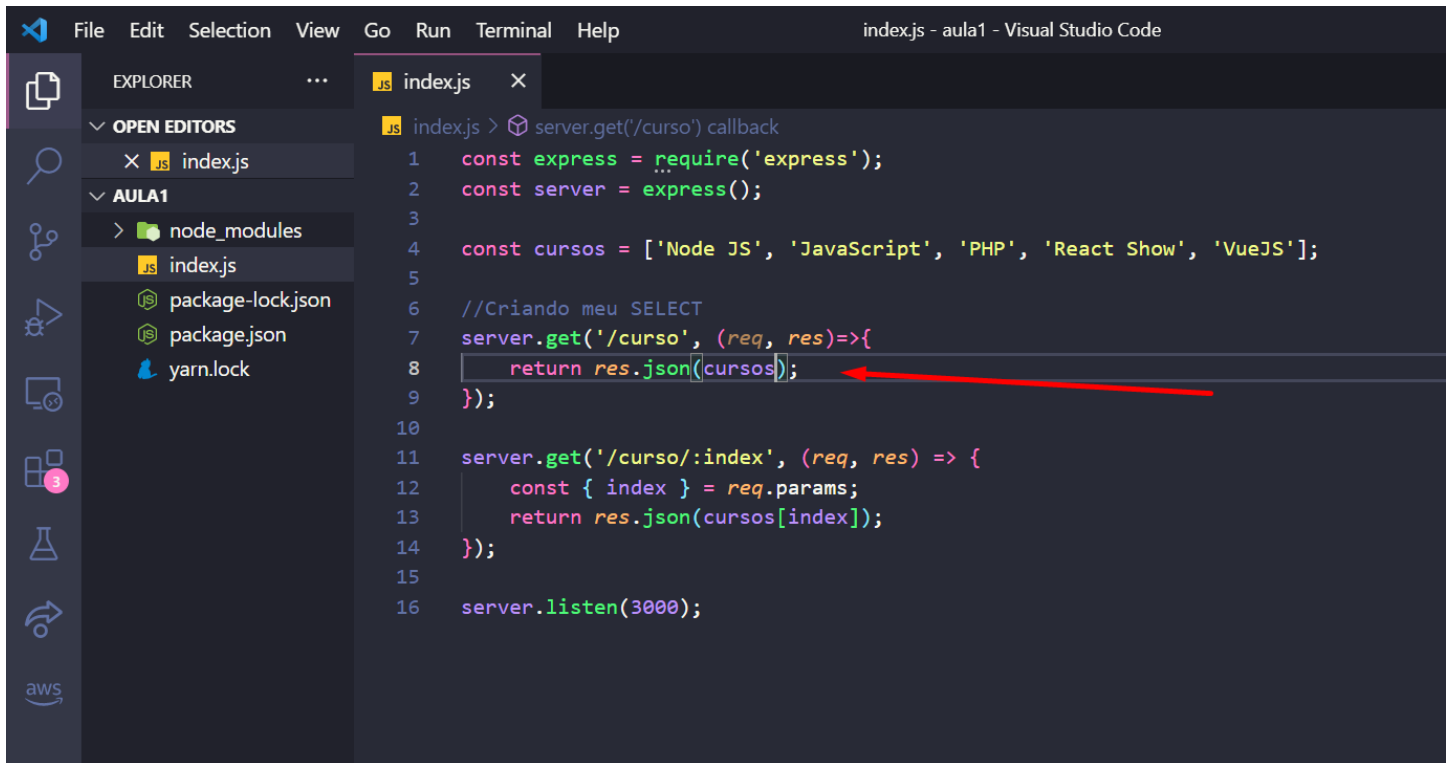
```
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
```


17 – Criamos um req e res para lidar tanto com as **REQ**uiições do Front-End quanto as **RES**postar do Back-end



```
index.js > server.get('/curso/:index') callback
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6  //Criando meu SELECT
7  server.get('/curso', (req, res)=>{
8
9  });
10
11 server.get('/curso/:index', (req, res) => {
12   const { index } = req.params;
13   return res.json(cursos[index]);
14 });
15
16 server.listen(3000);
```

18 – E dentro desse GET dizemos que se alguém fizer um get passando como rota / curso nós, no papel do back-end, devemos responder: `res.json(curso)`, ou seja, demos mandar em JSON todo nosso Array de curso.



```
index.js > server.get('/curso') callback
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6  //Criando meu SELECT
7  server.get('/curso', (req, res)=>{
8    return res.json(cursos);
9  });
10
11 server.get('/curso/:index', (req, res) => {
12   const { index } = req.params;
13   return res.json(cursos[index]);
14 });
15
16 server.listen(3000);
```

19 – Parabéns, você fez o primeiro passo do CRUD. Sua API agora consegue informar todo o conteúdo de um banco de dados.

The screenshot shows the ReqBin website, an online API testing tool. The interface includes a sidebar with various example requests, a main workspace for creating and testing API requests, and a right-hand panel for viewing the response details. The URL 'http://localhost:3000/cursos' is entered in the request field, and the response body shows a JSON array of technologies: ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS']. Red arrows highlight the URL input and the response content.

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

Online REST & SOAP API Testing Tool

File Generate Code Tools Share Generate Code App Mode

http://localhost:3000/cursos GET EXT Send Status: 200 (OK) Time: 21 ms Size: 0.05 kb

Authorization Content Headers Raw (2)

Bearer Token Basic Auth Custom No Auth

Token

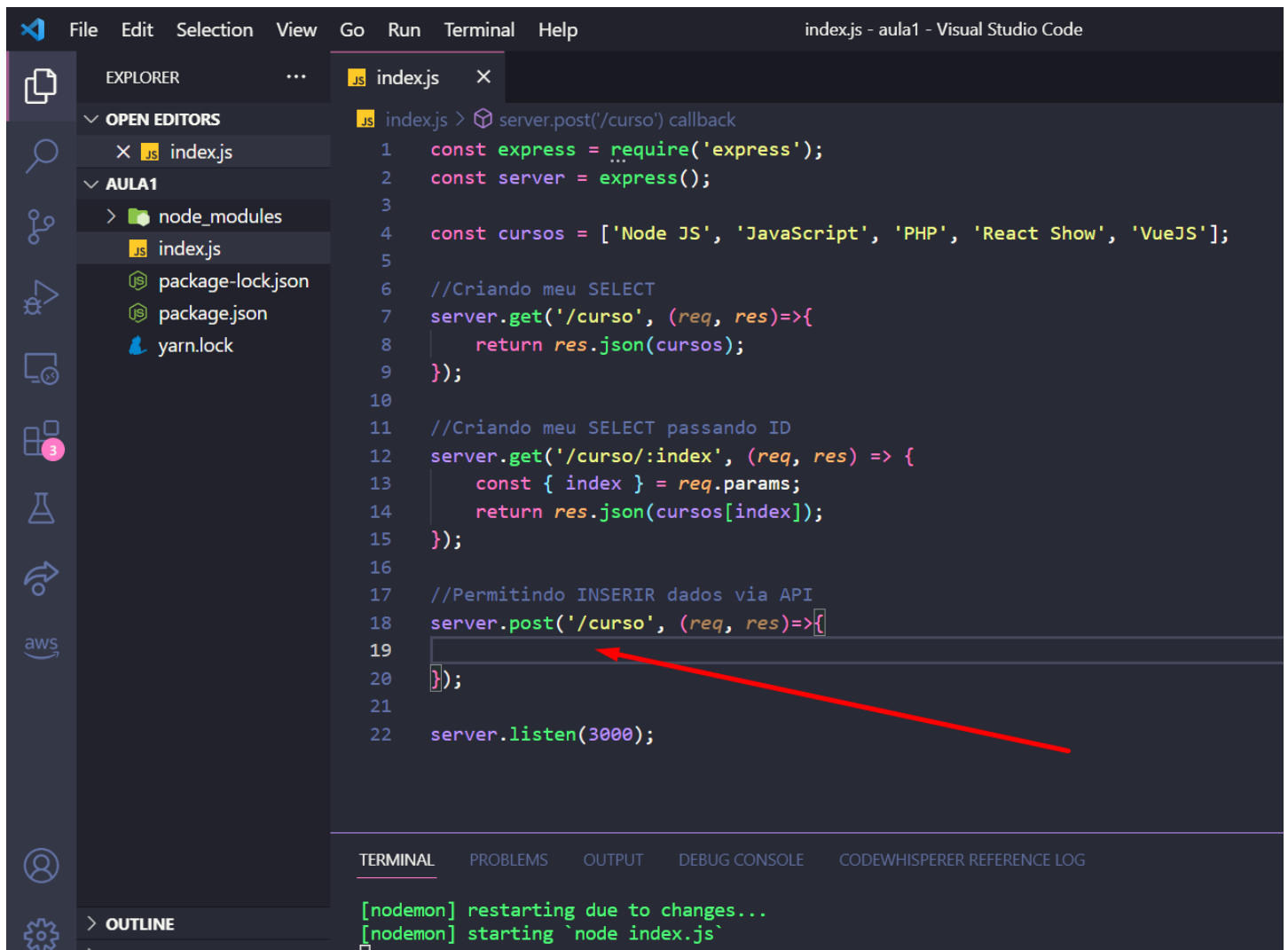
The authorization header will be automatically generated when you send the request. Read more about HTTP Authentication.

Content (1) Headers (8) Raw (10) JSON

["Node JS", "JavaScript", "PHP", "React Show", "VueJS"]

What is API?

20 – Vamos agora permitir que via API alguém adicione algo ao nosso banco de dados. Para isso via API nosso cliente deverá fazer um POST, portanto, adicione agora a possibilidade de tratar um POST (linha 17).

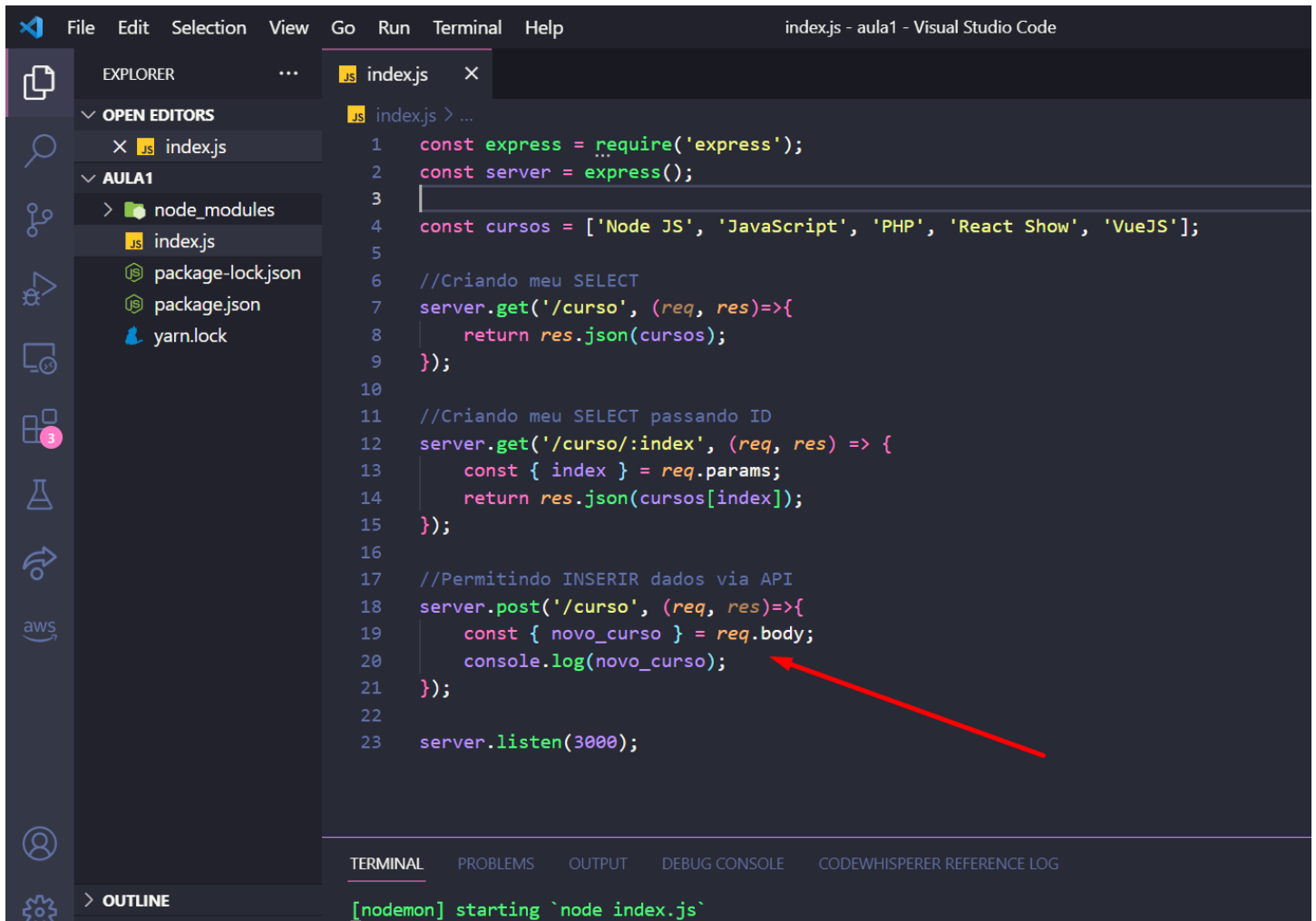


```
index.js > server.post('/curso') callback
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6  //Criando meu SELECT
7  server.get('/curso', (req, res)=>{
8    return res.json(cursos);
9  });
10
11 //Criando meu SELECT passando ID
12 server.get('/curso/:index', (req, res) => {
13   const { index } = req.params;
14   return res.json(cursos[index]);
15 });
16
17 //Permitindo INSERIR dados via API
18 server.post('/curso', (req, res)=>{
19
20 });
21
22 server.listen(3000);
```

TERMINAL

```
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
```

21 – Nossa regra será a seguinte, o cliente deve mandar dentro do Body do POST o nome do novo curso, iremos salvar esse valor enviado por ele pela const `novo_curso`. Ficando então `const { novo_curso } = req.body;` Vamos colocar também um `console.log` para ver se de fato está chegando até o nosso Back-end o valor do Body do POST.



```
1  const express = require('express');
2  const server = express();
3
4  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
5
6  //Criando meu SELECT
7  server.get('/curso', (req, res)=>{
8    return res.json(cursos);
9  });
10
11 //Criando meu SELECT passando ID
12 server.get('/curso/:index', (req, res) => {
13   const { index } = req.params;
14   return res.json(cursos[index]);
15 });
16
17 //Permitindo INSERIR dados via API
18 server.post('/curso', (req, res)=>{
19   const { novo_curso } = req.body;
20   console.log(novo_curso);
21 });
22
23 server.listen(3000);
```

Terminal output: [nodemon] starting `node index.js`

22 – Veja que nosso back-end gerou um erro tipo 500 e disse que não consegue destruturar a propriedade novo_curso.

The screenshot shows the ReqBin website interface. On the left is a sidebar with a list of example requests. The main area is titled "Online REST & SOAP API Testing Tool". It shows a request to `http://localhost:3000/curso` with a `POST` method and `EXT` type. The request body is `{ "novo_curso": "Laravel" }`. The response status is **500 (Internal Server Error)** with a time of 44 ms and size of 1.56 kb. The response content is an HTML error page with the title "Error" and a message: `TypeError: Cannot destructure property '#39;novo_curso' of #`. Two red arrows point to the request body and the error message. The bottom of the page has a section titled "What is API?" with a brief definition.

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

File ▾ ↗ Generate Code ▾ 🛠 Tools ▾ Share ↗ Generate Code ▾ App Mode

http://localhost:3000/curso POST EXT Send

Status: **500 (Internal Server Error)** Time: 44 ms Size: 1.56 kb

Authorization Content (1) Headers Raw (6) Content (13) Headers (9) Raw (20) HTML

JSON (application/json)

```
{ "novo_curso": "Laravel" }
```

```
<!DOCTYPE html>
<html lang="en">

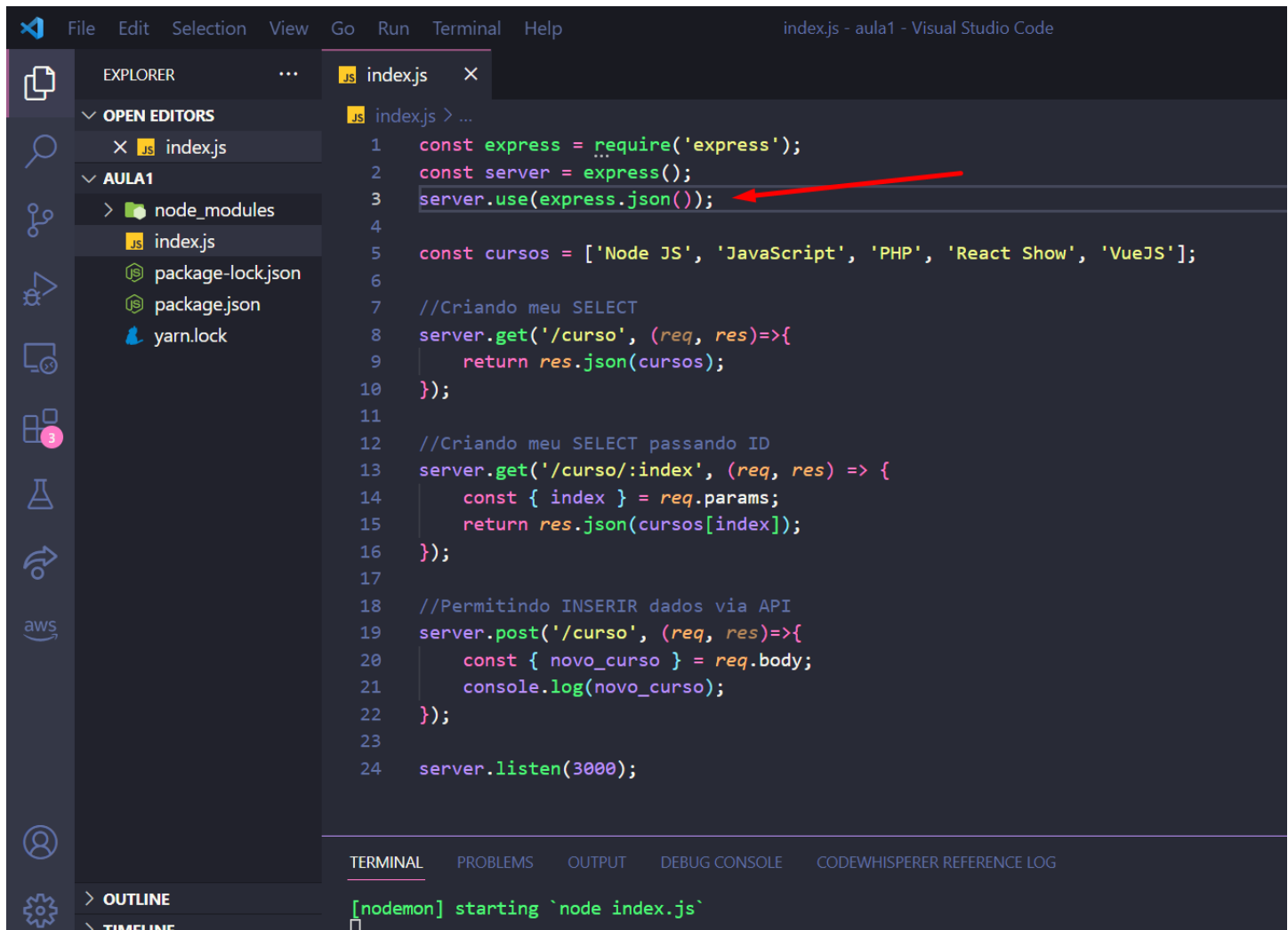
<head>
  <meta charset="utf-8">
  <title>Error</title>
</head>

<body>
  <pre>TypeError: Cannot destructure property '#39;novo_curso' of #
</body>
</html>
```

What is API?

API (Application Programming Interface) is a computing interface that defines how software components interact with each other. It is a way of programmatically interacting with a separate

23 – Esse erro ocorre devido nosso back-end até então nunca teve que manipular um JSON que chega do Front. Para resolver isso adicione `serve.use(express.json())`. Assim, via Express, nosso servidor consegue usar todos os recursos do JSON.



The screenshot shows the Visual Studio Code interface with the file `index.js` open. The Explorer sidebar on the left shows the project structure with `AULA1` containing `node_modules`, `index.js`, `package-lock.json`, `package.json`, and `yarn.lock`. The main editor displays the following JavaScript code:

```
1 const express = require('express');
2 const server = express();
3 server.use(express.json());
4
5 const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
6
7 //Criando meu SELECT
8 server.get('/curso', (req, res)=>{
9   return res.json(cursos);
10 });
11
12 //Criando meu SELECT passando ID
13 server.get('/curso/:index', (req, res) => {
14   const { index } = req.params;
15   return res.json(cursos[index]);
16 });
17
18 //Permitindo INSERIR dados via API
19 server.post('/curso', (req, res)=>{
20   const { novo_curso } = req.body;
21   console.log(novo_curso);
22 });
23
24 server.listen(3000);
```

A red arrow points to line 3, `server.use(express.json());`. The bottom status bar shows the terminal output: `[nodemon] starting `node index.js``.

24 – Vá no cliente e coloque o body sendo {"novo_curso":"Laravel"} e envie o POST ao nosso back-end.

The screenshot shows the ReqBin website interface for testing APIs. The main area is titled "Online REST & SOAP API Testing Tool". It includes a sidebar with various example requests, a central form for creating a request, and a right sidebar with an advertisement. The URL field contains "http://localhost:3000/curso", the method is set to "POST", and the body is set to "JSON (application/json)" with the content {"novo_curso":"Laravel"}. A "Send" button is visible. Red arrows highlight the URL, the "Send" button, and the body content.

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

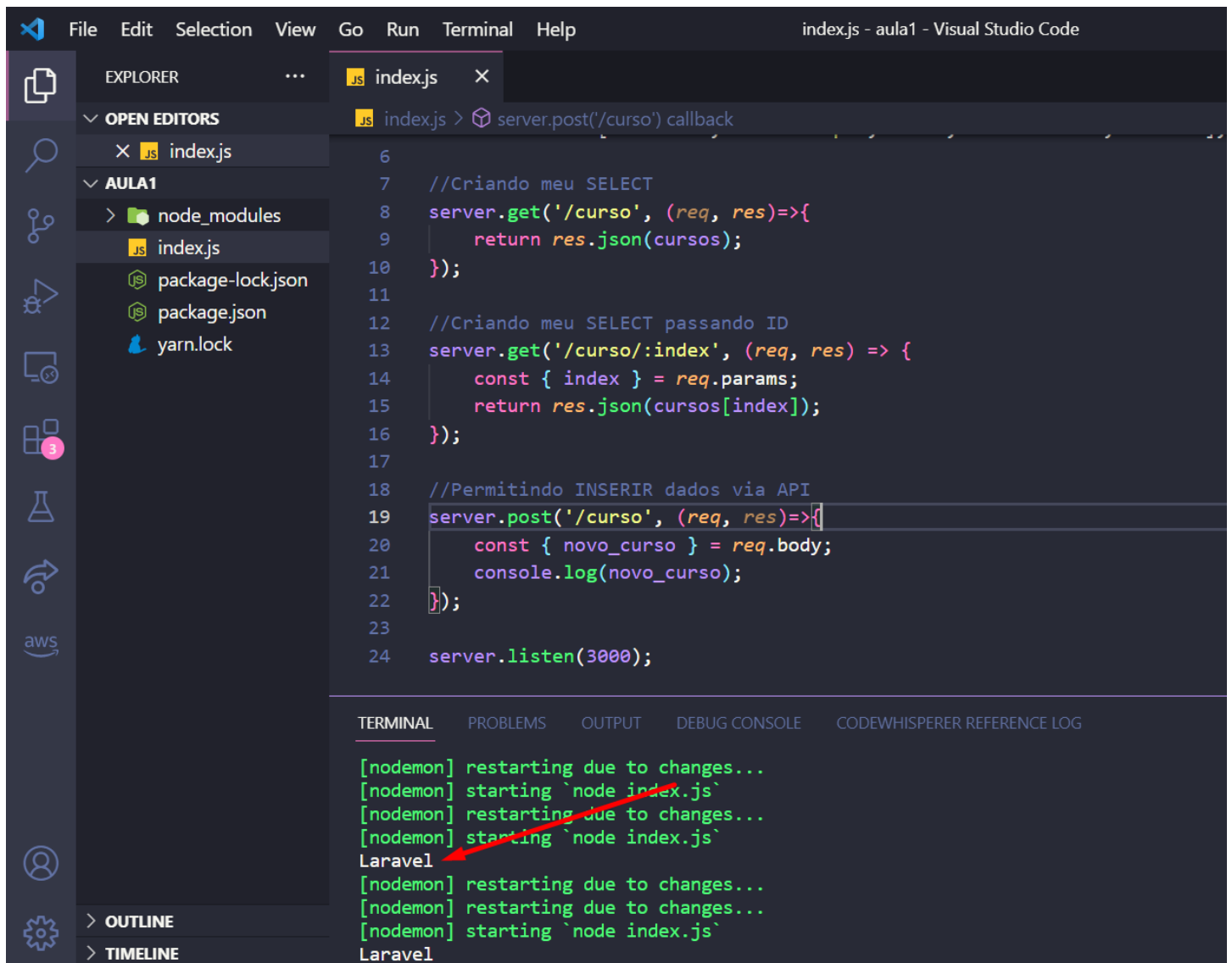
ReqBin Chrome Extension
Request timeout.

JSON (application/json)

```
{"novo_curso": "Laravel"}
```

What is API?

25 – Veja que agora, usando o console.log, podemos ver que de fato via body estamos recebendo o JSON enviado pelo cliente. Porém, não queremos ver no log mas sim adicionar dentro do nosso “banco de dados” já que estamos implementando um CRUD.



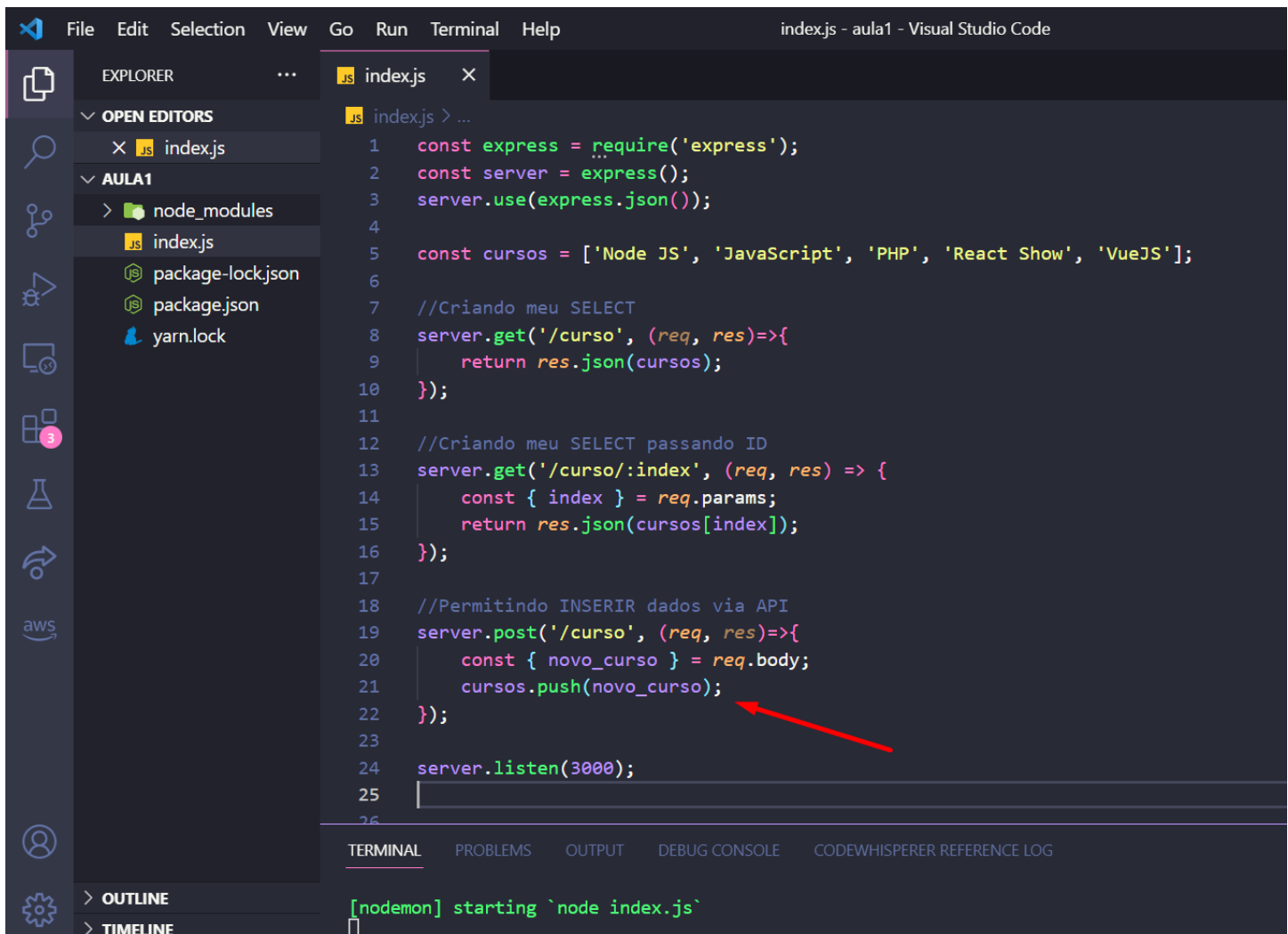
```
File Edit Selection View Go Run Terminal Help index.js - aula1 - Visual Studio Code

EXPLORER
OPEN EDITORS
  index.js
AULA1
  node_modules
  index.js
  package-lock.json
  package.json
  yarn.lock

6
7 //Criando meu SELECT
8 server.get('/curso', (req, res)=>{
9   return res.json(cursos);
10 });
11
12 //Criando meu SELECT passando ID
13 server.get('/curso/:index', (req, res) => {
14   const { index } = req.params;
15   return res.json(cursos[index]);
16 });
17
18 //Permitindo INSERIR dados via API
19 server.post('/curso', (req, res)=>{
20   const { novo_curso } = req.body;
21   console.log(novo_curso);
22 });
23
24 server.listen(3000);

TERMINAL
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Laravel
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Laravel
```

26 – Para de fato adicionar um novo elemento nesse nosso JSON array nos usamos o push. Pegamos o nome do array `cursos.push(novo_curso)`;



The screenshot shows the Visual Studio Code interface with a file named `index.js` open. The Explorer sidebar on the left shows the project structure with a folder named `AULA1` containing `index.js`, `package-lock.json`, `package.json`, and `yarn.lock`. The main editor displays the following JavaScript code:

```
1  const express = require('express');
2  const server = express();
3  server.use(express.json());
4
5  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
6
7  //Criando meu SELECT
8  server.get('/curso', (req, res)=>{
9    return res.json(cursos);
10 });
11
12 //Criando meu SELECT passando ID
13 server.get('/curso/:index', (req, res) => {
14   const { index } = req.params;
15   return res.json(cursos[index]);
16 });
17
18 //Permitindo INSERIR dados via API
19 server.post('/curso', (req, res)=>{
20   const { novo_curso } = req.body;
21   cursos.push(novo_curso);
22 });
23
24 server.listen(3000);
25
```

A red arrow points to the `cursos.push(novo_curso);` line on line 21. The bottom status bar shows the terminal output: `[nodemon] starting `node index.js``.

27 – Vamos agora adicionar o curso de C++ usando o POST. Perceba que o cliente não foi informado se o INSERT deu certo. Isso acontece pois em nosso back-end não configuramos a resposta apropriada do POST. É muito importante que o cliente que utiliza nossa API saiba se sua requisição deu certo.

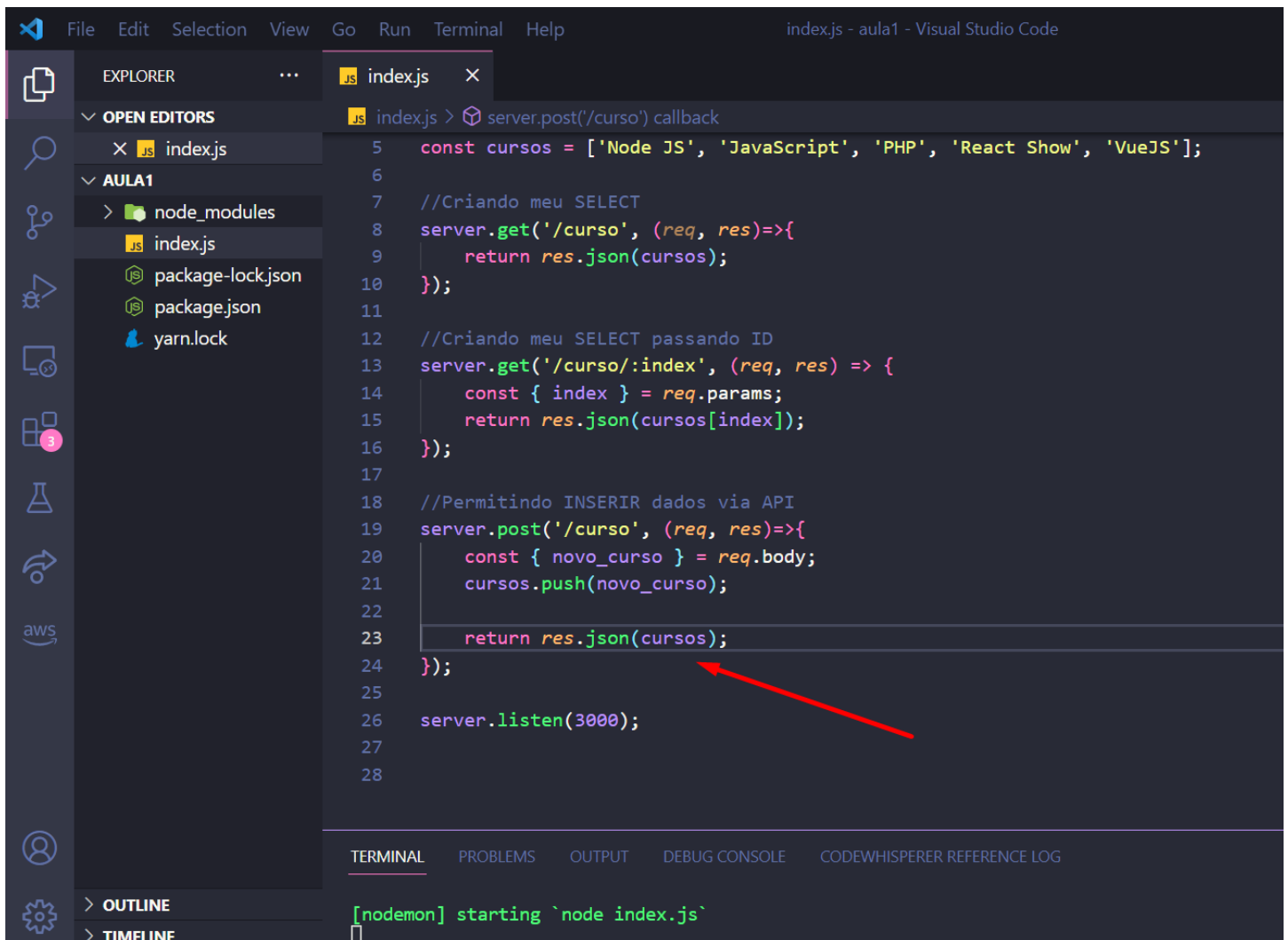
The screenshot shows the ReqBin website, an online REST & SOAP API testing tool. The interface includes a sidebar with various example requests, a main area for constructing and testing API requests, and a right sidebar with an advertisement for Sustagen Senior.

ReqBin Chrome Extension
Request timeout.

The main interface shows a request to `http://localhost:3000/curso` using the `POST` method. The request body is `{\"novo_curso\": \"C++\"}`. The response area, highlighted with a red box, displays the message `Request timeout.`

What is API?

28 – Adicionamos um return a função do POST e aqui podemos retornar todos os cursos, assim o cliente poderá ver o curso que acabou de adicionar.



```
File Edit Selection View Go Run Terminal Help index.js - aula1 - Visual Studio Code

EXPLORER
  OPEN EDITORS
    index.js
  AULA1
    node_modules
    index.js
    package-lock.json
    package.json
    yarn.lock

index.js
5  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
6
7  //Criando meu SELECT
8  server.get('/curso', (req, res)=>{
9    return res.json(cursos);
10 });
11
12 //Criando meu SELECT passando ID
13 server.get('/curso/:index', (req, res) => {
14   const { index } = req.params;
15   return res.json(cursos[index]);
16 });
17
18 //Permitindo INSERIR dados via API
19 server.post('/curso', (req, res)=>{
20   const { novo_curso } = req.body;
21   cursos.push(novo_curso);
22
23   return res.json(cursos);
24 });
25
26 server.listen(3000);
27
28

TERMINAL
[nodemon] starting `node index.js`
```

29 – Abrindo uma nova aba do Reqbin e dando um GET em /curso podemos ver a lista completa de cursos, incluindo os que foram adicionados pelo POST.

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

File ▾ ↗ Generate Code ▾ 🛠 Tools ▾ ⚙ Share ↗ ↗ Generate Code ▾ 🔄 App Mode

http://localhost:3000/curso GET ▾ EXT ▾ Send

Status: 200 (OK) Time: 4 ms Size: 0.06 kb

Authorization Content Headers Raw (2)

☒ Bearer Token ☐ Basic Auth ☐ Custom ☐ No Auth

Token

The authorization header will be automatically generated when you send the request. Read more about HTTP Authentication.

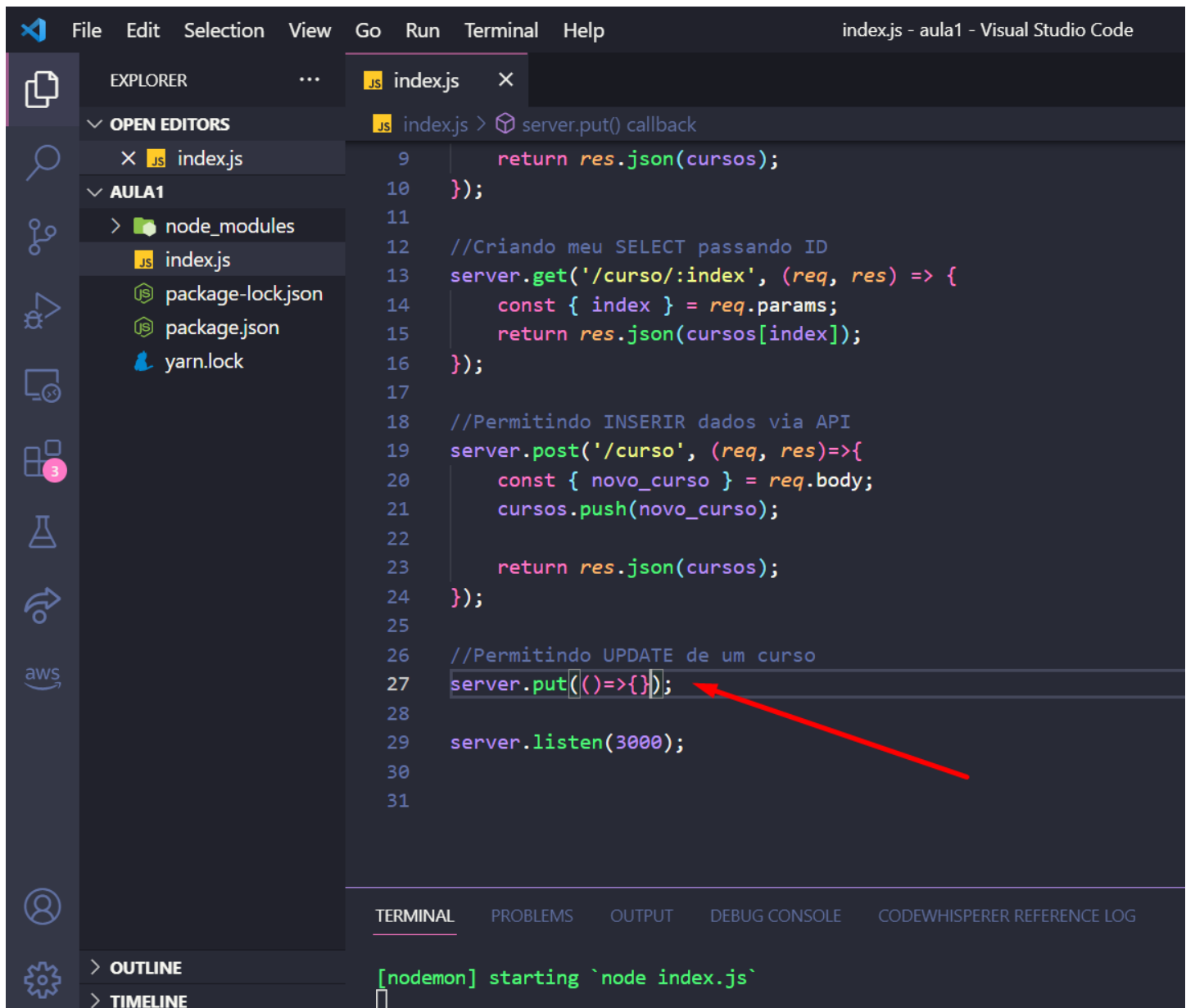
Content (1) Headers (8) Raw (10) JSON

```
[{"Node JS", "JavaScript", "PHP", "React Show", "VueJS", "C++"}]
```

What is API?

API (Application Programming Interface) is a computing interface that defines how software components interact with each other. It is a way of programmatically interacting with a separate software component or resource and expose functionality for internal or external use and testing. API defines what requests can be made, how they will be made and hides complexity from

30 – Vamos agora fazer o UPDATE do nosso CRUD. Adicione o server.put



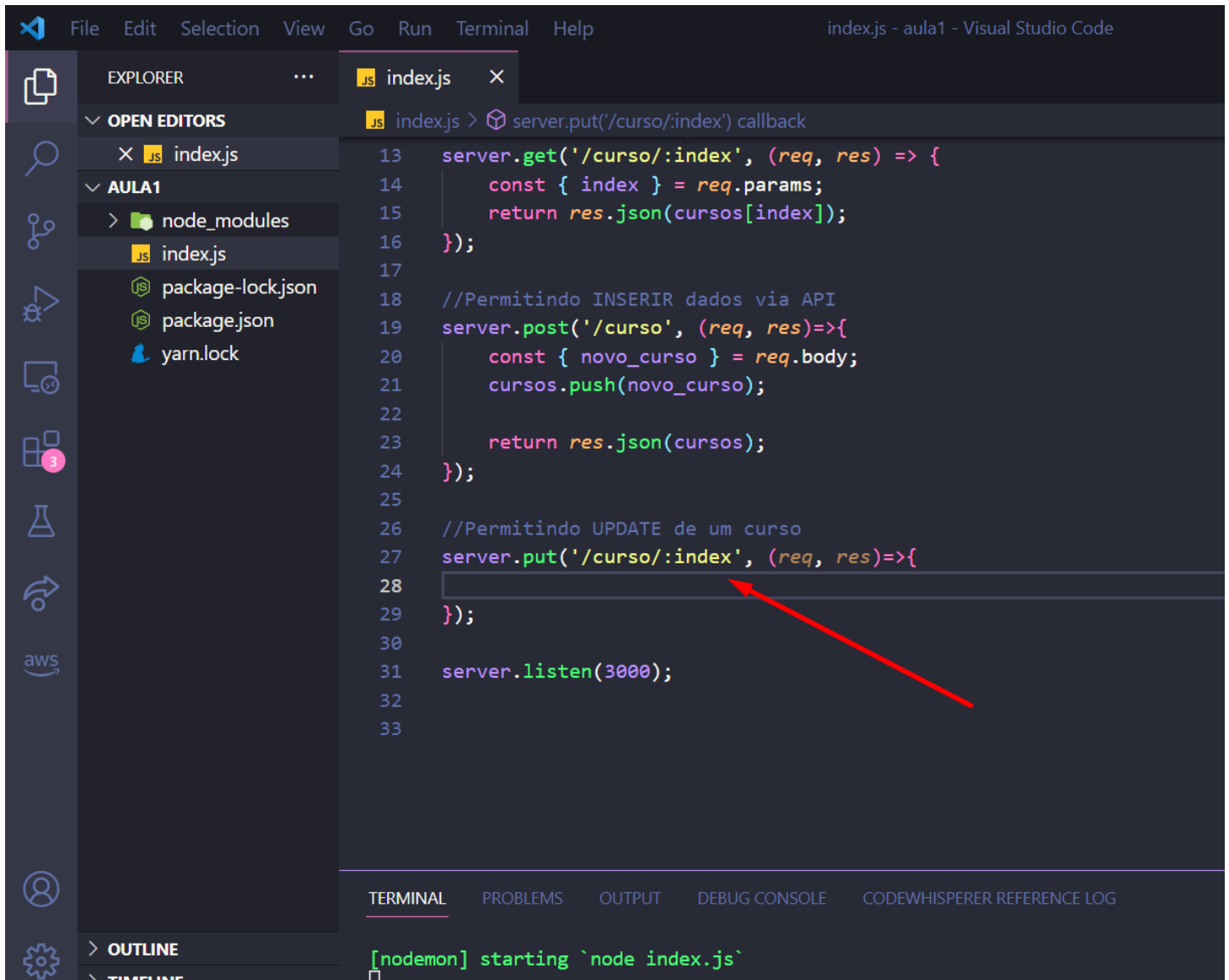
The screenshot shows the Visual Studio Code interface with the following components:

- Explorer Panel:** Displays the project structure under 'AULA1', including 'node_modules', 'index.js', 'package-lock.json', 'package.json', and 'yarn.lock'.
- Editor Panel:** Shows the 'index.js' file with the following code:

```
9      return res.json(cursos);
10    });
11
12    //Criando meu SELECT passando ID
13    server.get('/curso/:index', (req, res) => {
14      const { index } = req.params;
15      return res.json(cursos[index]);
16    });
17
18    //Permitindo INSERIR dados via API
19    server.post('/curso', (req, res)=>{
20      const { novo_curso } = req.body;
21      cursos.push(novo_curso);
22
23      return res.json(cursos);
24    });
25
26    //Permitindo UPDATE de um curso
27    server.put(()=>{});
28
29    server.listen(3000);
30
31
```

A red arrow points to the `server.put(()=>{});` line on line 27, indicating the addition of the PUT endpoint.
- Terminal Panel:** Shows the command `[nodemon] starting `node index.js``.

31 – Para nosso back-end saber qual curso deve atualizar, temos que passar qual a posição (index) dentro do array que ele deve atualizar. Portanto a rota desse PUT será /curso/:index



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left displaying the project structure under 'AULA1', including 'node_modules', 'index.js', 'package-lock.json', 'package.json', and 'yarn.lock'. The main editor area shows the 'index.js' file with the following code:

```
13 server.get('/curso/:index', (req, res) => {
14     const { index } = req.params;
15     return res.json(cursos[index]);
16 });
17
18 //Permitindo INSERIR dados via API
19 server.post('/curso', (req, res)=>{
20     const { novo_curso } = req.body;
21     cursos.push(novo_curso);
22
23     return res.json(cursos);
24 });
25
26 //Permitindo UPDATE de um curso
27 server.put('/curso/:index', (req, res)=>{
28
29 });
30
31 server.listen(3000);
32
33
```

A red arrow points to the route path `/curso/:index` in the `server.put` function definition on line 27. The status bar at the bottom indicates '[nodemon] starting `node index.js`'.

32 – Dentro do PUT devemos receber via params qual o id que o usuário quer atualizar, exemplo, se ele quer atualizar o curso “Node JS” ele deve passar a rota /curso/0 ou se quiser atualizar o curso “JavaScript” a rota será /curso/1 e assim por diante. Iremos salvar esse parametro passado na rota dentro do index. Depois o conteúdo do body (req.body) iremos salvar em curso, depois adicionamos a nova atualização dentro do array utilizando `cursos[index] = curso`. Porfim, retornamos ao cliente com a lista de todos os cursos para ele saber se deu certo o UPDATE.

```
File Edit Selection View Go Run Terminal Help index.js - aula1 - Visual Studio Code

EXPLORER
  OPEN EDITORS
    index.js
  AULA1
    node_modules
    index.js
    package-lock.json
    package.json
    yarn.lock

index.js
21  cursos.push(novo_curso);
22
23  return res.json(cursos);
24  });
25
26  //Permitindo UPDATE de um curso
27  server.put('/curso/:index', (req, res)=>{
28    const { index } = req.params;
29    const { curso } = req.body;
30
31    cursos[index] = curso;
32
33    return res.json(cursos);
34  });
35
36  server.listen(3000);
37
38

TERMINAL
[nodemon] starting `node index.js`
```


33 – Veja que agora nós podemos atualizar qualquer valor dentro do array passando na URL o id e dentro do body do PUT o novo valor.

Online REST & SOAP API Testing Tool

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

File Generate Code Tools Share Generate Code App Mode

http://localhost:3000/curso/0 PUT EXT Send Status: 200 (OK) Time: 333 ms Size: 0.06 kb

Authorization Content (1) Headers Raw (6) Content (1) Headers (8) Raw (10) JSON

JSON (application/json)

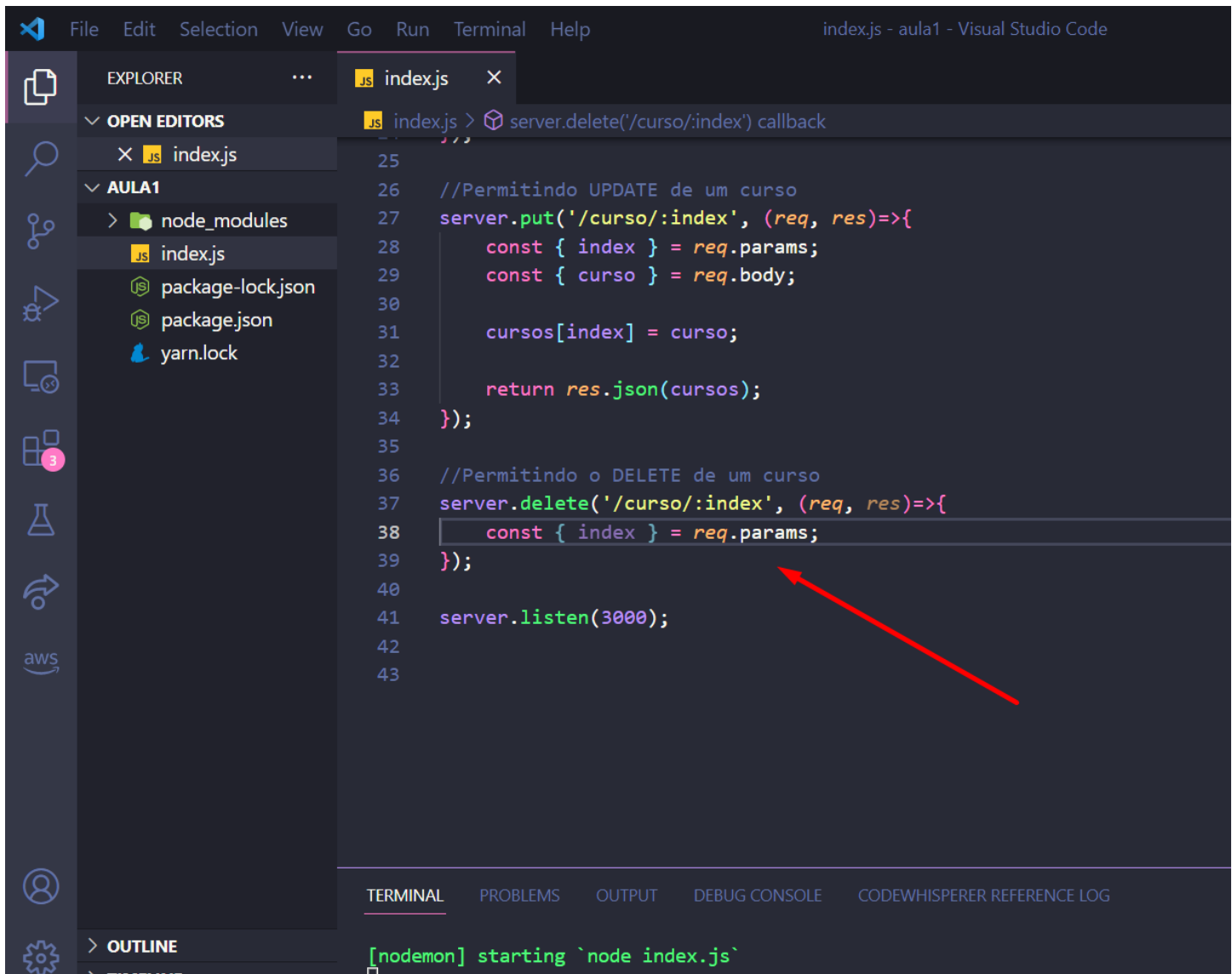
```
{\"curso\": \"Node JS Avançado\"}
```

[\"Node JS Avançado\", \"JavaScript\", \"PHP\", \"React Show\", \"VueJS\"]

What is API?

API (Application Programming Interface) is a...

34 – Vamos agora implementar o DELETE de nosso CRUD. Crie o server.delete. Veja se será similar ao PUT onde também vamos usar o id como parametro para que nosso back-end saiba qual ele deverá curso ele deve deletar.



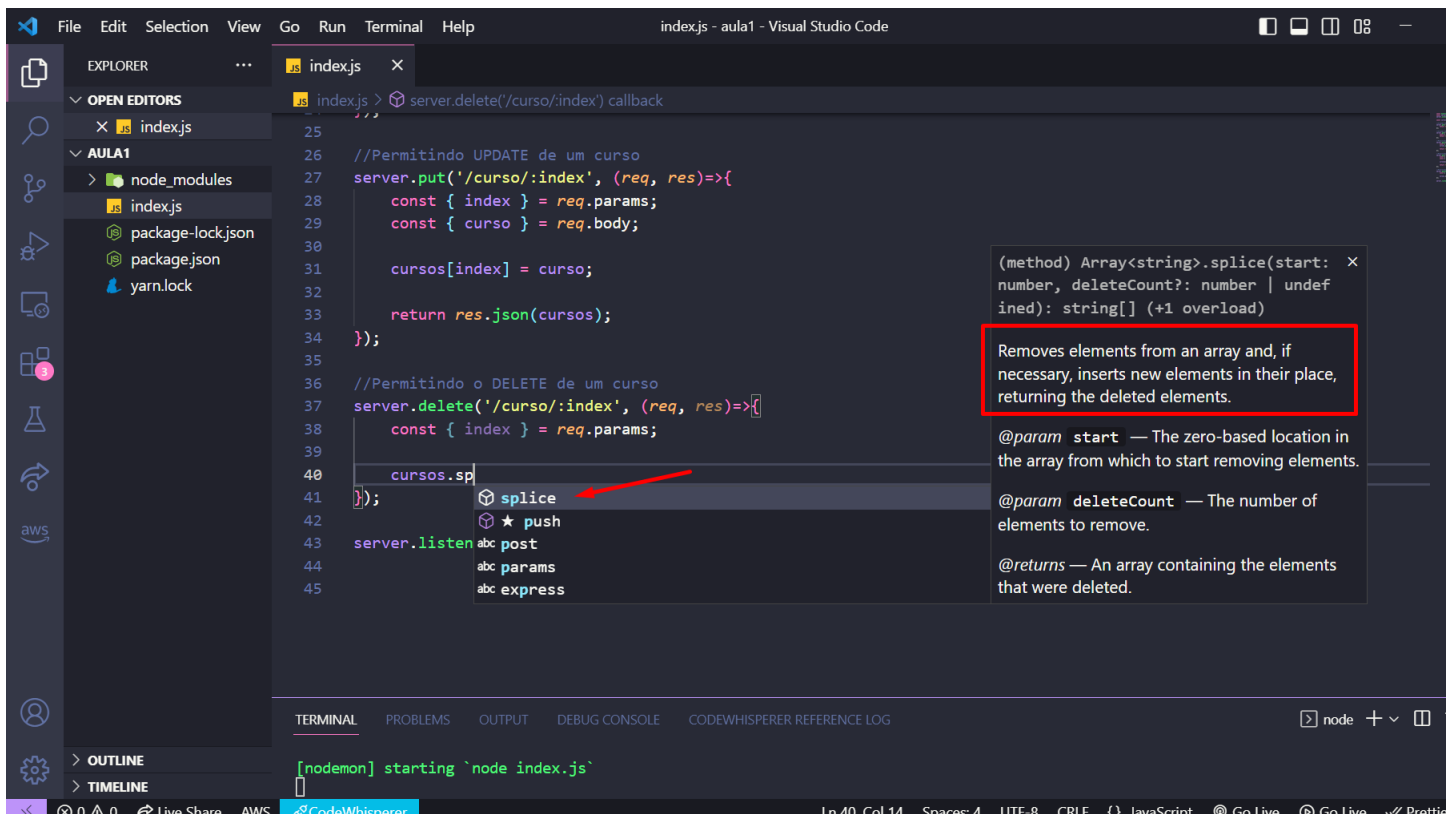
```
index.js - aula1 - Visual Studio Code

EXPLORER
  OPEN EDITORS
    index.js
  AULA1
    node_modules
    index.js
    package-lock.json
    package.json
    yarn.lock

index.js
25
26 //Permitindo UPDATE de um curso
27 server.put('/curso/:index', (req, res)=>{
28   const { index } = req.params;
29   const { curso } = req.body;
30
31   cursos[index] = curso;
32
33   return res.json(cursos);
34 });
35
36 //Permitindo o DELETE de um curso
37 server.delete('/curso/:index', (req, res)=>{
38   const { index } = req.params;
39 });
40
41 server.listen(3000);
42
43

TERMINAL
[nodemon] starting `node index.js`
```

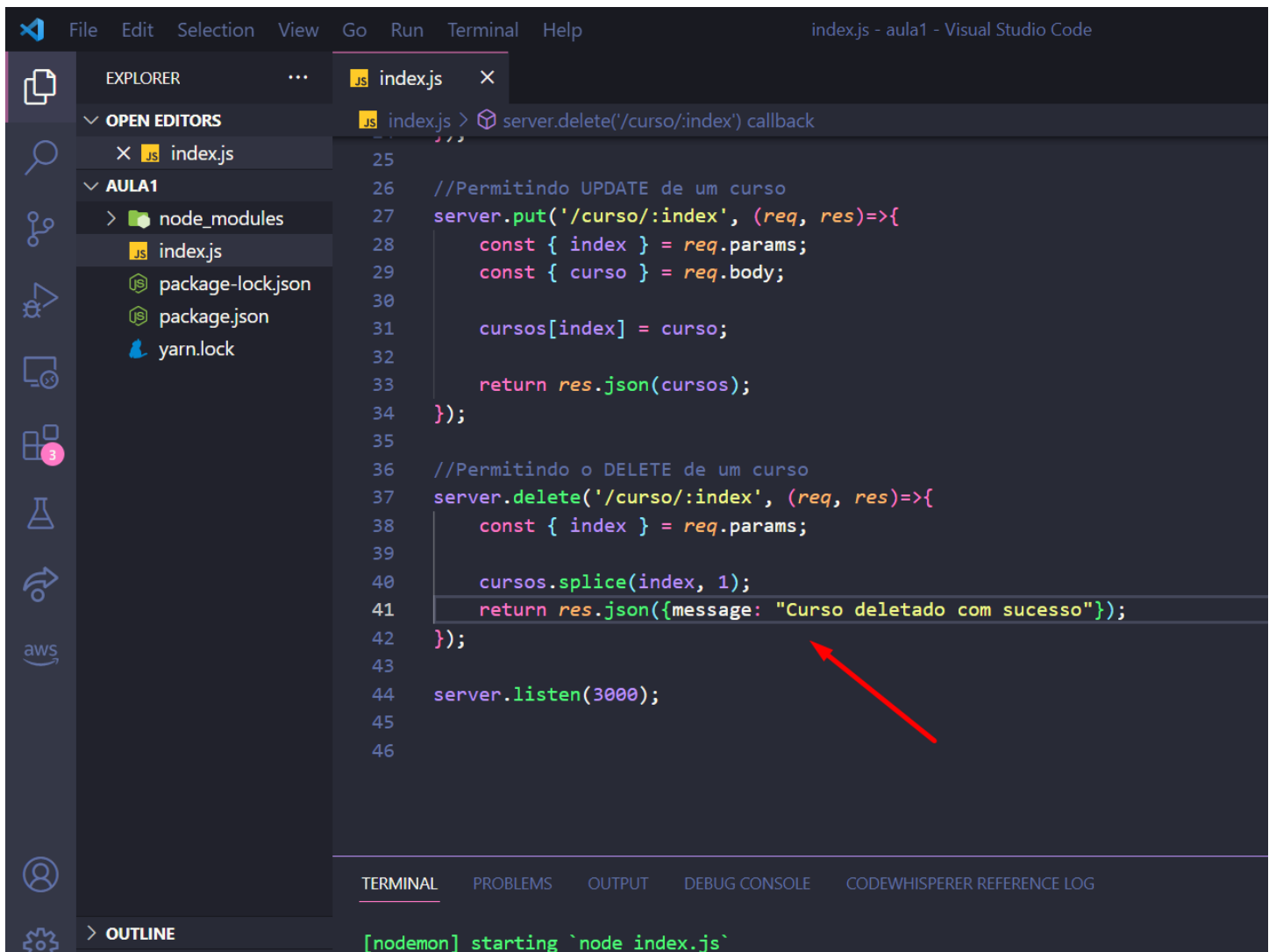
35 – Vamos usar a função SPLICE do JavaScript para deletar, já que o que queremos é apenas deletar um valor dentro do ARRAY.



36 – Ao fazer o DELETE via API, veja que ele retorna o código 200 e mostra todos os cursos do nosso banco, demonstrando que o valor foi excluído.

The screenshot displays the ReqBin online REST & SOAP API Testing Tool interface. The URL bar shows `http://localhost:3000/curso/0`. The method is set to `DELETE`. The response status is `Status: 200 (OK)`, with a time of `565 ms` and a size of `0.04 kb`. The response content is displayed in the `Content (1)` tab, showing the JSON array `["JavaScript", "PHP", "React Show", "VueJS"]`. The interface includes a sidebar with various API examples, a top navigation bar with language options (Curl, Python, JavaScript, Node.js, PHP, Java, JSON, XML), and a right sidebar with an advertisement for Tokio Marine.

37 – Outra possibilidade é informar uma mensagem do tipo “Curso deletado com sucesso” ao invés de retornar todos os cursos.



The image shows a Visual Studio Code editor window with a file named `index.js` open. The file contains JavaScript code for a REST API. The code defines two endpoints: a PUT endpoint for updating a course and a DELETE endpoint for deleting a course. The DELETE endpoint is highlighted with a red arrow pointing to the response message.

```
25
26 //Permitindo UPDATE de um curso
27 server.put('/curso/:index', (req, res)=>{
28   const { index } = req.params;
29   const { curso } = req.body;
30
31   cursos[index] = curso;
32
33   return res.json(cursos);
34 });
35
36 //Permitindo o DELETE de um curso
37 server.delete('/curso/:index', (req, res)=>{
38   const { index } = req.params;
39
40   cursos.splice(index, 1);
41   return res.json({message: "Curso deletado com sucesso"});
42 });
43
44 server.listen(3000);
45
46
```

The terminal at the bottom shows the command `[nodemon] starting 'node index.js'`.

38 – Ficando assim a resposta na API

REQLBIN

EXAMPLES SAVED

POST Request Example

Bearer Token Auth Example

HTML Form POST Example

GET Request Example

REST API POST Example

PUT Request Example

Auth Bearer Header Example

POST API Example

POST JSON Example

JSON Payload Example

Content-Length Example

REST API GET Example

Send Cookies Example

POST XML Example

JSON Response Example

POST JSON with Bearer Token

Curl POST Request Example

Curl GET Request Example

Curl POST JSON Example

Curl POST Body Example

Curl Basic Auth Example

Curl Bearer Token Example

Curl Download File Example

Curl Ignore SSL Checks

JavaScript POST Example

JavaScript GET Example

JavaScript Fetch JSON

JavaScript Array Join Example

Online REST & SOAP API Testing Tool

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

File

</> Generate Code

Tools

http://localhost:3000/curso/3

DELETE

EXT

Send

Share

</> Generate Code

App Mode

Status: **200 (OK)** Time: **72 ms** Size: **0.04 kb**

Authorization

Content

Headers

Raw (2)

Bearer Token

Basic Auth

Custom

No Auth

Token

The authorization header will be automatically generated when you send the request. Read more about HTTP Authentication.

Content (3)

Headers (8)

Raw (10)

JSON

{

"message": "Curso deletado com sucesso"

}

ADVERTISEMENT

Adobe

Estudantes economizam mais de 60% na Adobe Creative Cloud.

Compartilhe sua visão com o mundo.

Comprar agora

MATA PENA

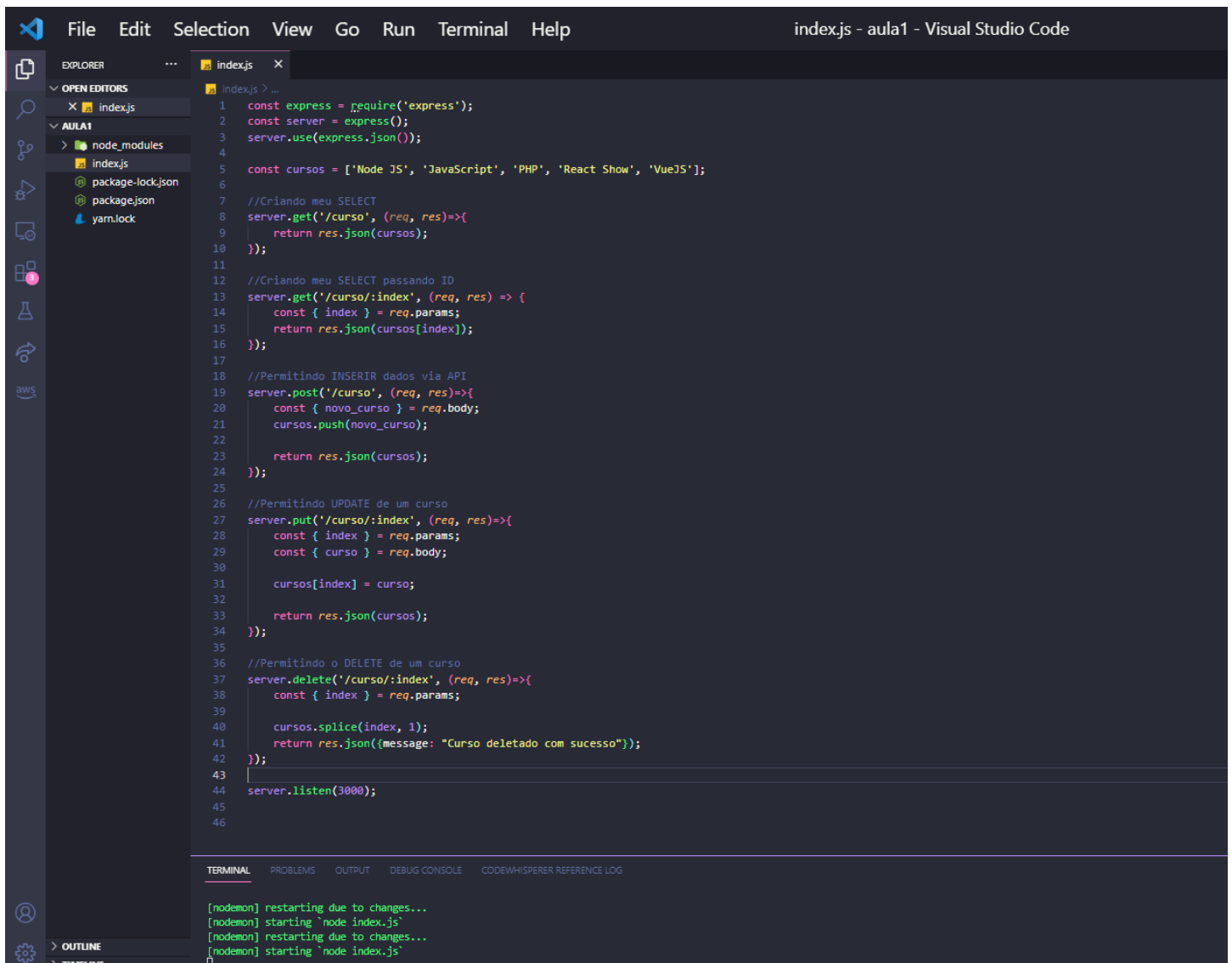
ECOOPREENDEDORA

report this ad

39 – Comon seria a documentação do CRUD da API que acabamos de implementar.

Selecionando todos os cursos	GET /curso
Selecionando um curso via ID	GET /curso/id
Adicionando um curso	POST /curso
	Body: {"novo_curso": "Laravel"}
Atualizando um curso	PUT /curso/id
	Body: {"curso": "Node JS Avançado"}
Deletando um curso	DELETE /curso/id

40 – Código completo da API



```
1  const express = require('express');
2  const server = express();
3  server.use(express.json());
4
5  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
6
7  //Criando meu SELECT
8  server.get('/curso', (req, res)=>{
9      return res.json(cursos);
10 });
11
12 //Criando meu SELECT passando ID
13 server.get('/curso/:index', (req, res) => {
14     const { index } = req.params;
15     return res.json(cursos[index]);
16 });
17
18 //Permitindo INSERIR dados via API
19 server.post('/curso', (req, res)=>{
20     const { novo_curso } = req.body;
21     cursos.push(novo_curso);
22
23     return res.json(cursos);
24 });
25
26 //Permitindo UPDATE de um curso
27 server.put('/curso/:index', (req, res)=>{
28     const { index } = req.params;
29     const { curso } = req.body;
30
31     cursos[index] = curso;
32
33     return res.json(cursos);
34 });
35
36 //Permitindo o DELETE de um curso
37 server.delete('/curso/:index', (req, res)=>{
38     const { index } = req.params;
39
40     cursos.splice(index, 1);
41     return res.json({message: "Curso deletado com sucesso"});
42 });
43
44 server.listen(3000);
45
46
```

TERMINAL

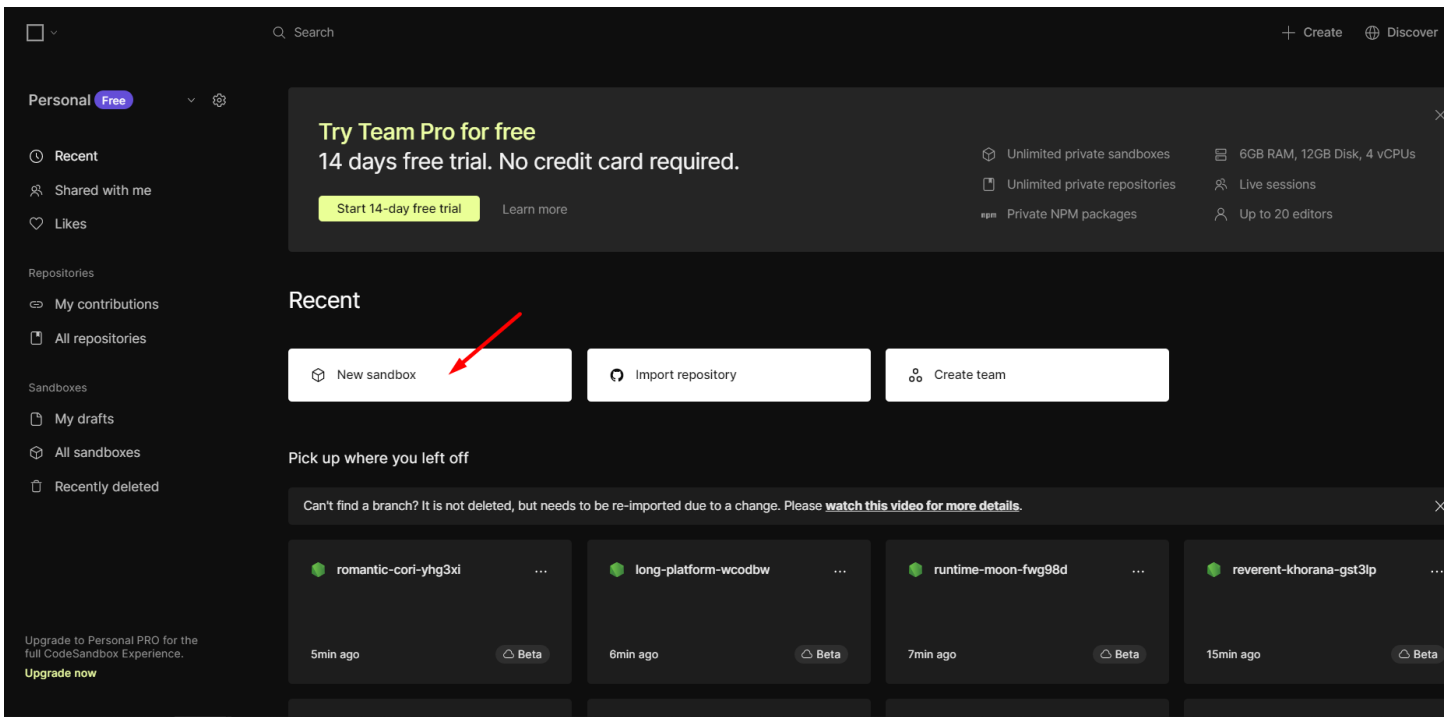
```
[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'
[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'
```


Exercícios

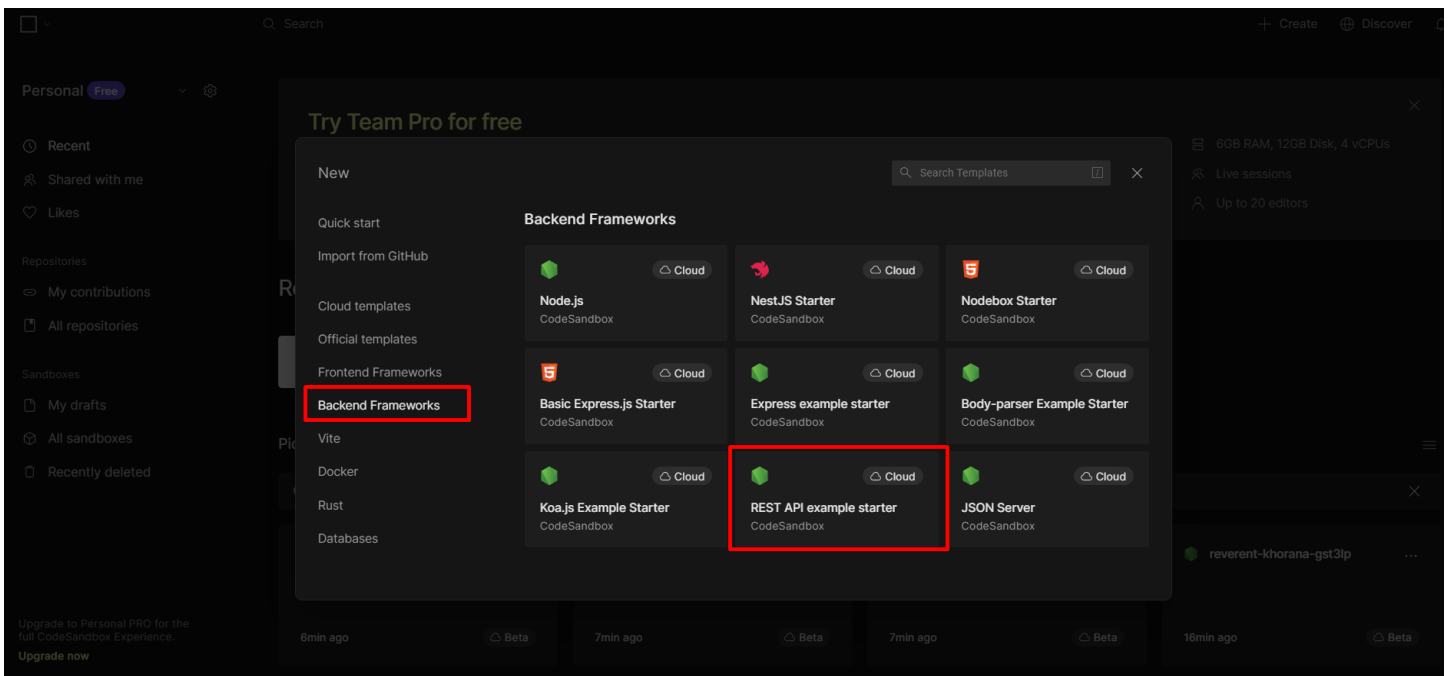
Vamos fazer o deploy do nosso servidor?

01 - Acesse: <https://codesandbox.io/>

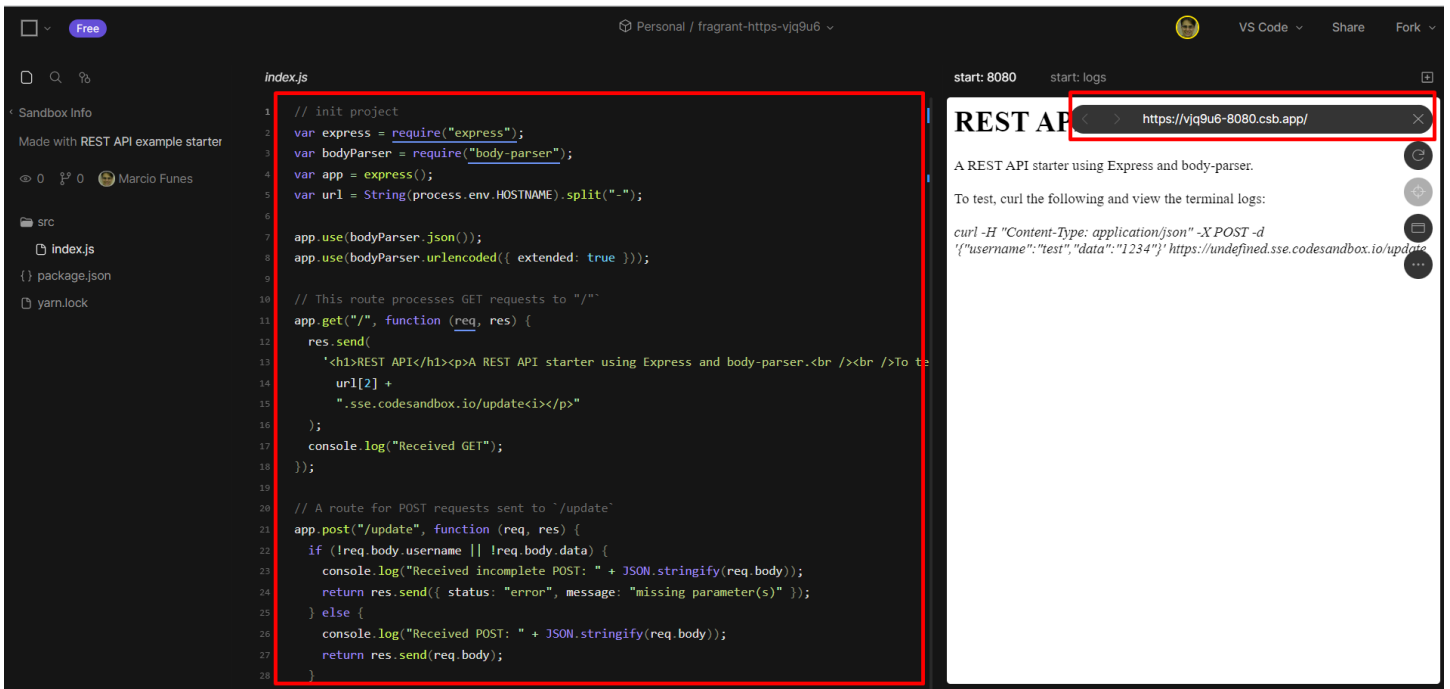
02 - Vá em New sandbox



03 – Vá em Backend Frameworks → REST API exemple starter



04 – Aqui você deverá implementar o CRUD do mesmo modo que fizemos localmente. Veja que no canto superior direito temos o endpoint da nossa API online. Isso significa que podemos testar nossa API de modo público.



05 – Teste sua API pública, faça todas as operações GET, POST, PUT e DELETE no POSTMAN e veja se consegue os mesmo resultados que localmente.

